



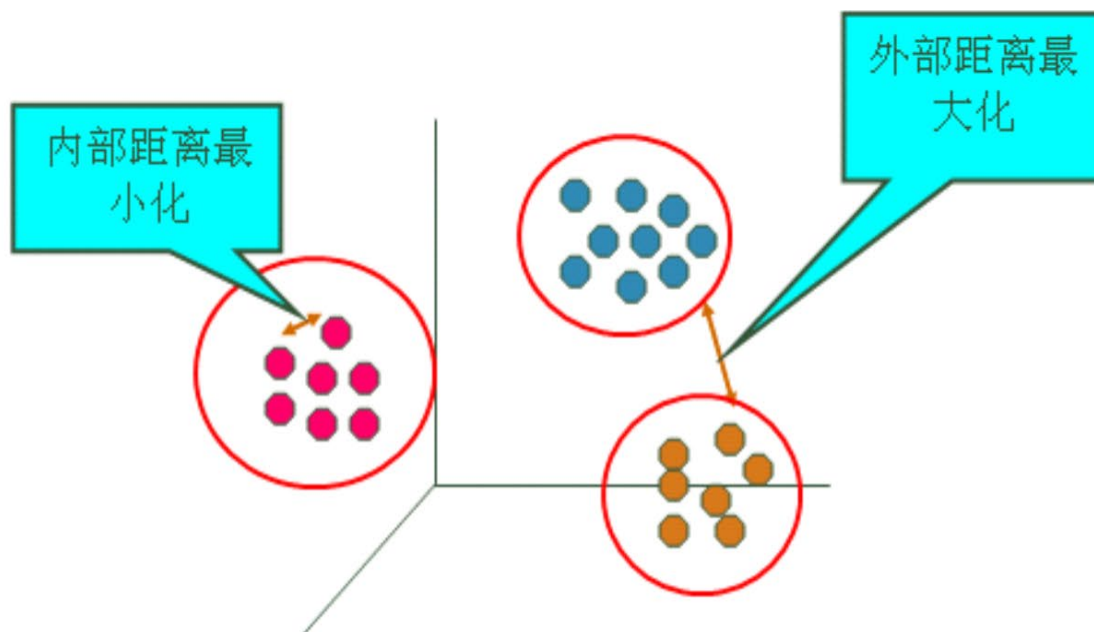
聚类分析

2026/5/7

- 定义及相关概念
- 主要方法
- 评价指标
- 案例

聚类分析——概念

- 与分类不同，聚类分析是在没有给定划分类别的情况下，根据数据相似程度进行样本分组的一种方法。
- 与分类模型需要使用有类标记样本构成的训练数据不同，聚类模型可以建立在无类标记的数据上，是一种非监督的学习算法。
- 聚类的输入是一组未被标记的样本，聚类根据数据自身的距离或相似度将他们划分为若干组，划分的原则是组内样本最小化而组间（外部）距离最大化：



聚类分析——数据点的数值类型

- 数据点的类型可分为：
 - 欧氏 (Euclidean)
 - 非欧
- 这二者在数据的表示以及处理上有较大的不同：
 - 怎样来表示cluster ?
 - 怎样来计算相似度?

- 欧氏空间：
 - 取各个数据点的平均值 (centroid)
- 非欧空间
 - 取某个处于最中间的点
 - 取若干个最具代表性的点 (clustroid)
 -

聚类类型

- 分离良好的聚类
- 基于中心的聚类
- 连续簇
- 基于密度的聚类
- 属性或概念
- 由目标函数描述

聚类分析——客户分群案例

- 案例实现：

部分餐饮客户的消费行为特征数据如下表，根据这些数据将客户分类成不同客户群，并评价这些客户群的价值。

ID	R	F	M
1	37	4	579
2	35	3	616
3	25	10	394
4	52	2	111
5	36	7	521
6	41	5	225
7	56	3	118
8	37	5	793
9	54	2	111
10	5	18	1086

最近一次消费 (Recency)

消费频率 (Frequency)

消费金额 (Monetary)

聚类分析——K-Means聚类算法

- 案例实现：

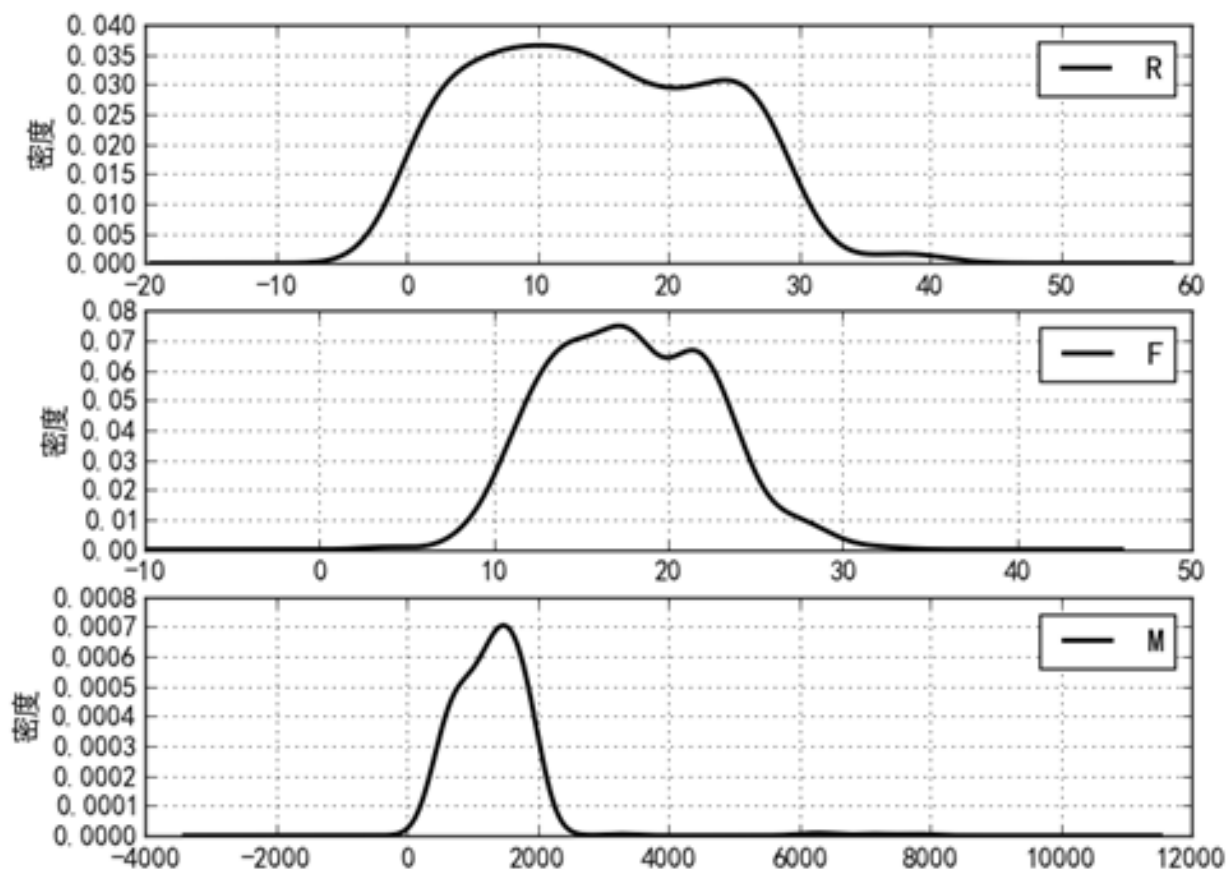
采用K-Means聚类算法，将客户聚类3组。输出结果如下表：

分群类别		分群1	分群2	分群3
样本个数		340	560	40
样本个数占比		12.77%	65.53%	21.70%
聚类中心	R	-0.16295092	-0.14785515	3.45505486
	F	1.11672177	-0.65689153	-0.29565357
	M	0.39557542	-0.27225103	0.44912342

聚类分析——客户分群案例

● 案例实现：

以下是用Pandas和Matplotlib绘制的不同客户分群的概率密度函数图，通过这些图能直观地比较不同客户群的价值。分群 1 的概率密度函数图：

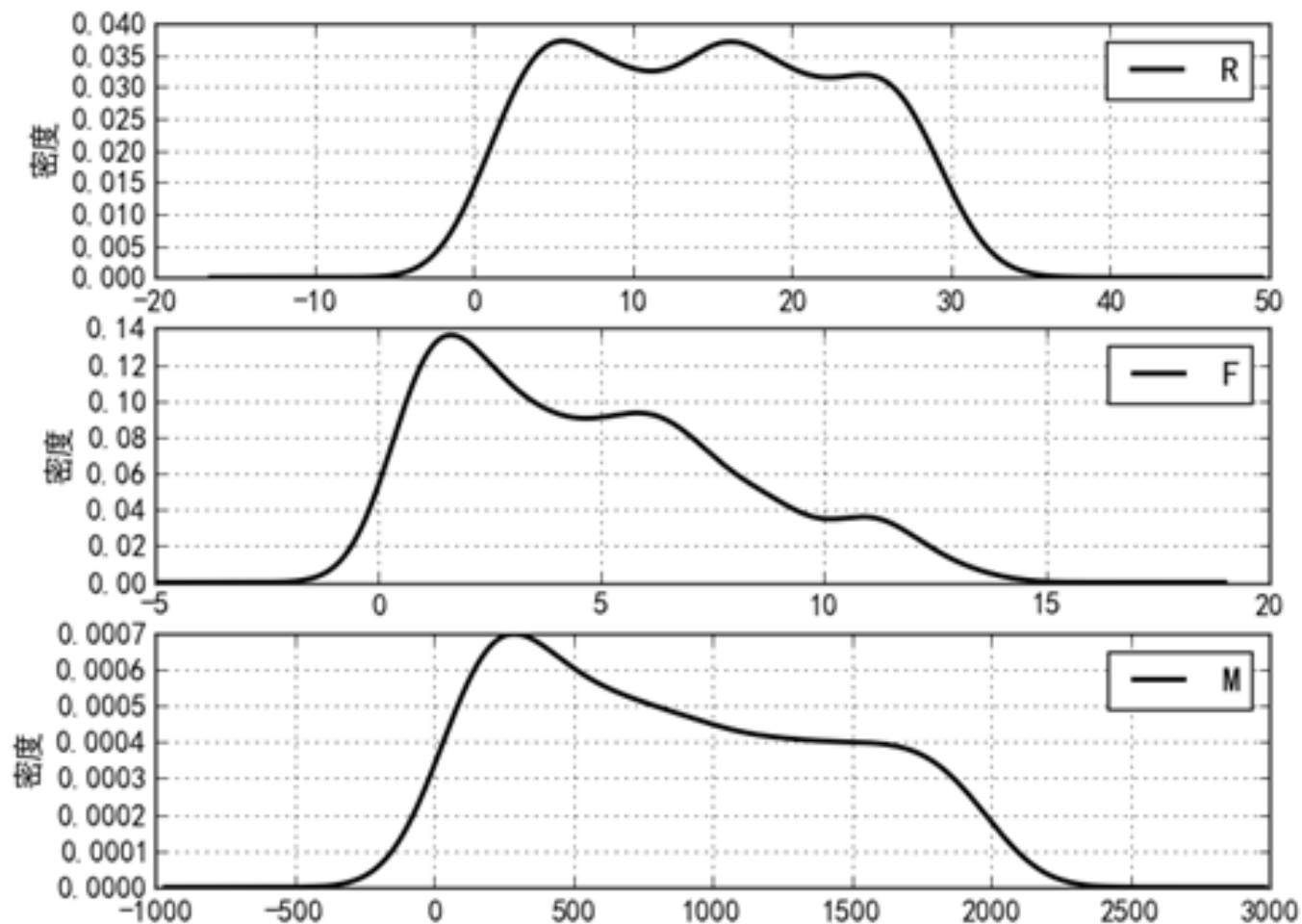


分群1特点：R间隔相对较小，主要集中在0~30天之间；消费次数集中在10~25次；消费金额在500~2000。

聚类分析——客户分群案例

● 案例实现：

分群 2 的概率密度函数图：

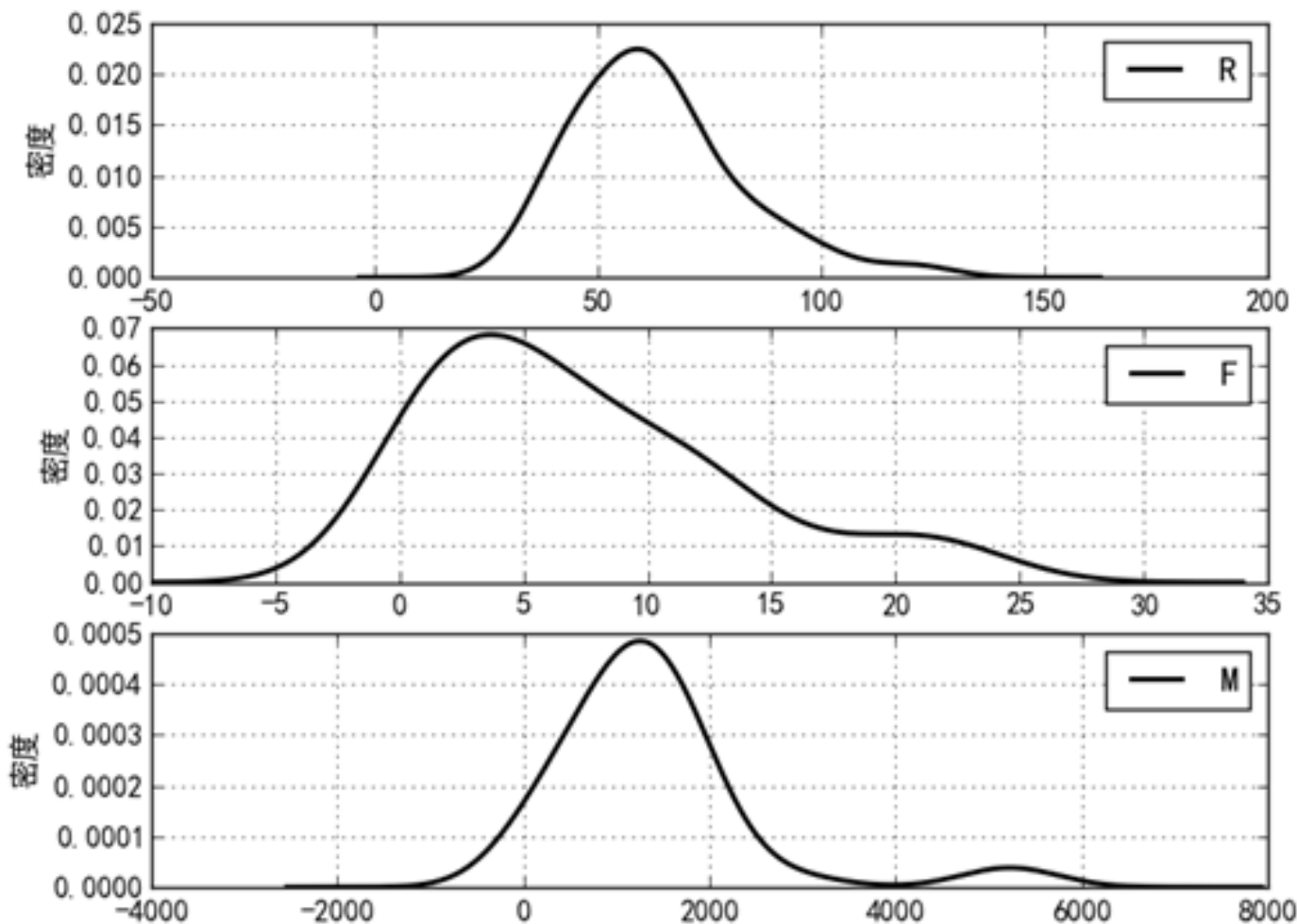


分群2特点：R间隔分布在0~30天之间；消费次数集中在0~12次；消费金额在0~1800。

聚类分析——客户分群案例

● 案例实现：

分群 3 的概率密度函数图：



分群3特点：R间隔相对较大，间隔分布在30~80天之间；消费次数集中在0~15次；消费金额在0~2000。

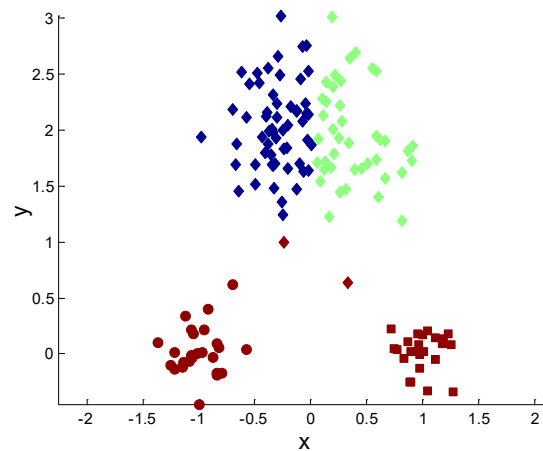
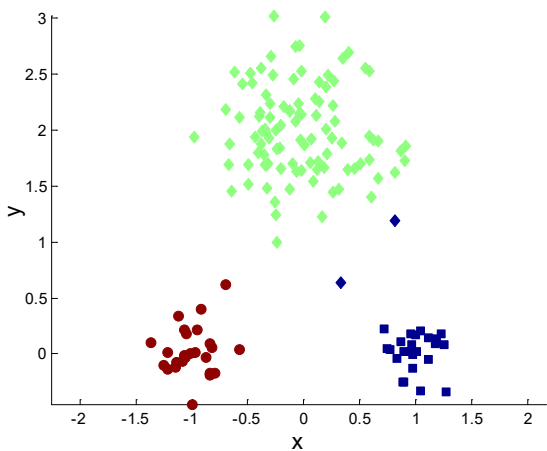
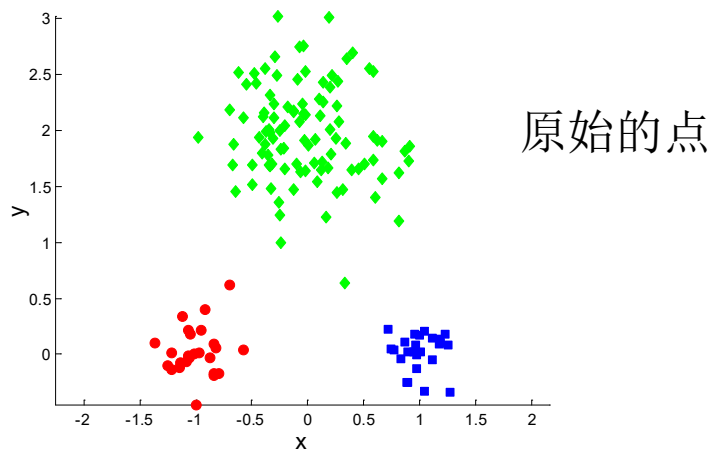
聚类分析——客户分群案例

● 案例实现：

客户价值分析：

- ❖ 分群1特点：R间隔相对较小，主要集中在0~30天之间；消费次数集中在10~25次；消费金额在500~2000。
- ❖ 分群2特点：R间隔分布在0~30天之间；消费次数集中在0~12次；消费金额在0~1800。
- ❖ 分群3特点：R间隔相对较大，间隔分布在30~80天之间；消费次数集中在0~15次；消费金额在0~2000。
- ❖ 对比分析：分群1时间间隔较短，消费次数多，而且消费金额较大，是高消费**高价值人群**。分群2的时间间隔、消费次数和消费金额处于中等水平，代表着**一般客户**。分群3的时间间隔较长，消费次数较少，消费金额也不是特别高，是**价值较低的客户群体**。

聚类算法评价——不同的聚类结果



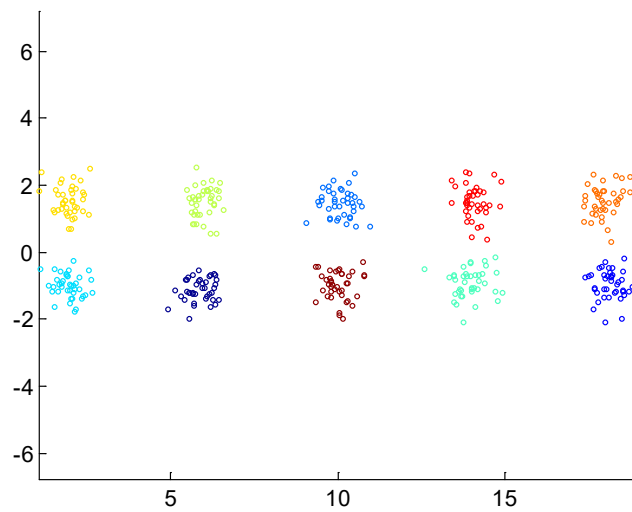
聚类算法评价——内部评价方法

内部评价方法：没有数据标签，不考虑外部信息的情况下直接对聚类结果进行评价

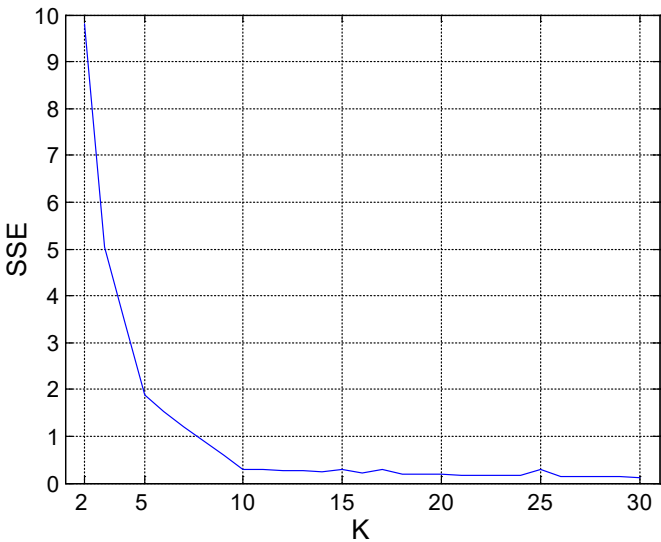
1) SSE (Sum of Squared Errors)

$$\sum_{l=1}^k \sum_{i=1}^n u_{il} d^2(x_i, q_l)$$

SSE有利于比较两个聚类或两个簇（平均SSE）
也可以用来估计聚类的数量



符号	含义
k	聚类簇的个数
q_l	第 l 个簇中心
x	对象（样本）
n	数据集中样本的个数
U	$n \times k$ 划分矩阵， $w_{il} = 1$ 表示第 i 个样本属于第 l 个簇
d(.,.)	距离



缺点：随着k增加，SSE趋向减少。

聚类分析——内部评价方法

2) 轮廓系数 (Silhouette Coefficient)

衡量对象和所属簇之间的相似度——即内聚性 (cohesion)。当把它与其他簇做比较，就称为分离性 (separation)。该对比通过 silhouette 值来实现，后者在 $[-1, 1]$ 范围内。Silhouette 值接近 1，说明对象与所属簇之间有密切联系；反之则接近 -1。若某模型中的一个数据簇，生成的基本是比较高的 silhouette 值，说明该模型是合适、可接受的。

计算方法：

- 1) 计算样本 i 到同簇其他样本的平均距离 $a(i)$ 。 $a(i)$ 越小，说明样本 i 越应该被聚类到该簇。将 $a(i)$ 称为样本 i 的簇内不相似度。簇 C 中所有样本的 $a(i)$ 均值称为簇 C 的簇不相似度。
- 2) 计算样本 i 到其他某簇 $C(j)$ 的所有样本的平均距离 $b(ij)$ ，称为样本 i 与簇 $C(j)$ 的不相似度。定义为样本 i 的簇间不相似度： $b(i) = \min \{b_{i1}, b_{i2}, \dots, b_{ik}\}$ ， $b(i)$ 越大，说明样本 i 越不属于其他簇。
- 3) 根据样本 i 的簇内不相似度 $a(i)$ 和簇间不相似度 $b(i)$ ，定义样本 i 的轮廓系数：

4) 判断：

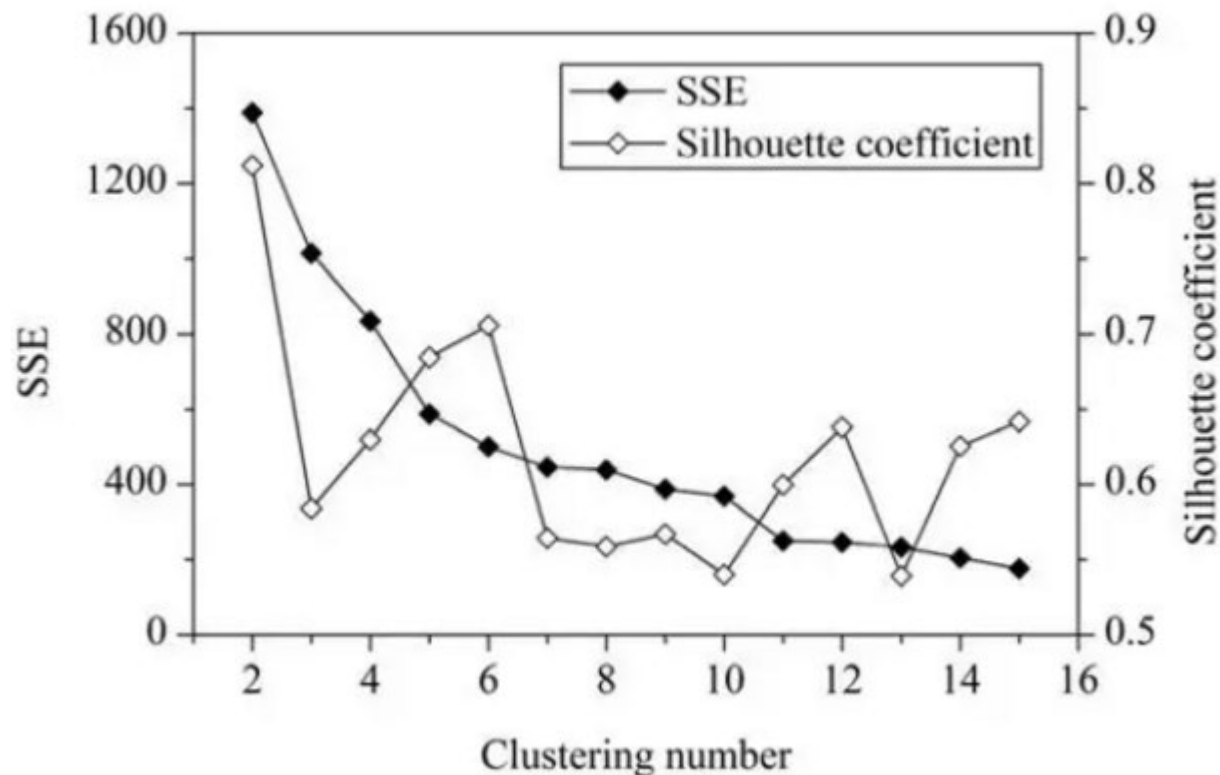
- $s(i)$ 接近 1，则说明样本 i 聚类合理
- $s(i)$ 接近 -1，则说明样本 i 更应该分类到另外的簇
- 若 $s(i)$ 近似为 0，则说明样本 i 在两个簇的边界上

$$s(i) = \frac{b(i) - a(i)}{\max \{a(i), b(i)\}} \quad s(i) = \begin{cases} 1 - \frac{a(i)}{b(i)}, & a(i) < b(i) \\ 0, & a(i) = b(i) \\ \frac{b(i)}{a(i)} - 1, & a(i) > b(i) \end{cases}$$

聚类分析——内部评价方法

轮廓系数 (Silhouette Coefficient)

计算所有样本 $s(i)$ 的均值



聚类分析——内部评价方法

3) CH系数 (Calinski-Harabaz Index)

对于聚类模型来说,我们希望聚类结果为相同类别之间的数据距离越近越好,而不同类别之间的数据距离越远越好。

CH指标通过计算类中各点与类中心的距离平方和来度量类内的紧密度,通过计算各类中心点与数据集中心点距离平方和来度量数据集的分离度,CH指标由分离度与紧密度的比值得到。从而,CH越大代表着类自身越紧密,类与类之间越分散,即更优的聚类结果。

$$CH(k) = \frac{B_k}{W_k} * \frac{n - k}{k - 1}$$

B_k 是类间散度, W_k 是类内散度。

$$W_k = \sum_{l=1}^k \sum_{x \in C_l} \|x - q_l\|_2^2$$
$$B_k = \sum_{l=1}^k n_l \|q_l - \bar{x}\|_2^2$$

q_l 是第 l 个簇 C_l 的中心, n_l 是第 l 个簇包含的样本个数, $\bar{x}$ 是整个数据集的中心。

聚类算法评价——外部评价方法

外部评价方法：利用数据标签对聚类结果进行评价

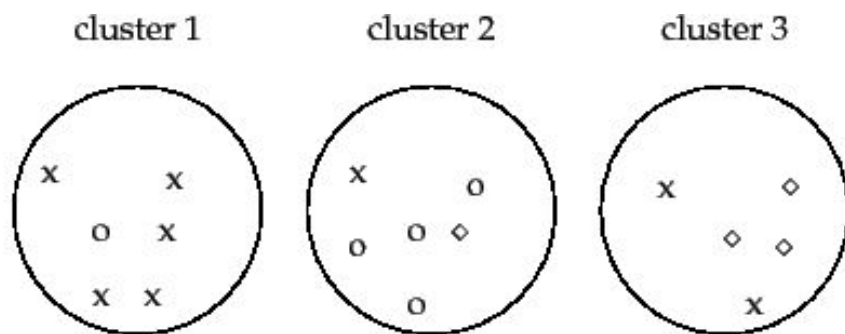
1) Purity

假设 n 为总样本个数， $\Omega = \{w_1, \dots, w_k\}$ 表示 k 个簇， $C = \{c_1, \dots, c_J\}$ 表示真实类别划分。

$$\text{Purity}(\Omega, C) = \frac{1}{n} \sum_{l=1}^k \max_j |w_l \cap c_j|$$

$\text{Purity} \in [0,1]$ ，越大表示聚类结果越好

缺点：无法用于权衡聚类质量与簇个数之间的关系



在「cluster 1」中「class x」数目最多为 5，在「cluster 2」中「class o」数目最多为 4，在「cluster 3」中「class ◇」数目最多为 3。因此：

$$\text{Purity} = \frac{5+4+3}{17} \approx 0.7059$$

聚类算法评价——外部评价方法

2) RI (Rand index , 兰德指数)

假设有n个样本，则有n(n-1)/2个集合对

TP: 同一类的样本被正确分到同一个簇

TN: 不同类的样本被正确分到不同簇

FP: 不同类的样本被错误分到同一个簇

FN: 同一类的样本被错误分到不同簇

Rand Index度量的正确的百分比

$$RI = \frac{TP+TN}{TP+TN+FP+FN} = \frac{TP+TN}{C_n^2}$$

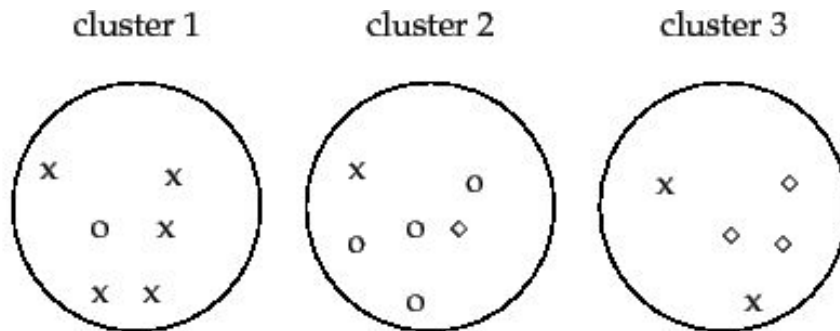
$$TP = C_5^2 + C_4^2 + C_3^2 + C_2^2 = 20$$

$$TN = C_5^1(C_5^1+C_3^1) + C_1^1(C_2^1+C_5^1) + C_1^1C_3^1 + C_4^1C_5^1 + C_1^1C_2^1 = 72$$

$$TP + FP = C_6^2 + C_6^2 + C_5^2 = 40$$

$$TP + FP + FP + FN = C_{17}^2 = 136$$

$$RI = (20+72)/136=0.68$$



	Same cluster	Different clusters
Same class	TP=20	FN=24
Different classes	FP=20	TN=72

聚类算法评价——外部评价方法

3) F_β

但是 RI 给 FP 和 FN 相同的权重,「分开相似的样本」的后果有时比将「不相似的样本放在同一类」更严重,因此我们可以 FN 罚更多,通过取 F_β 中的 β 大于 1, 此时实际上也相当于赋予召回率更大的权重(把好瓜全部挑上)。

$$P(\text{Precision}) = \frac{TP}{TP+FP} = \frac{20}{40} = 0.5$$

$$R(\text{Recall}) = \frac{TP}{TP+FN} = \frac{20}{44} \approx 0.455$$

$$F_\beta = \frac{(1 + \beta^2)P \times R}{\beta^2 P + R} \quad F_1 \approx 0.48 \quad F_5 \approx 0.456$$

	Same cluster	Different clusters
Same class	TP=20	FN=24
Different classes	FP=20	TN=72

- 基于划分的聚类
- 基于密度的聚类
- 谱聚类

Python聚类工具

Method name	Parameters	Scalability	Usecase	Geometry (metric used)
K-Means	number of clusters	Very large n_samples, medium n_clusters with MiniBatch code	General-purpose, even cluster size, flat geometry, not too many clusters	Distances between points
Affinity propagation	damping, sample preference	Not scalable with n_samples	Many clusters, uneven cluster size, non-flat geometry	Graph distance (e.g. nearest-neighbor graph)
Mean-shift	bandwidth	Not scalable with n_samples	Many clusters, uneven cluster size, non-flat geometry	Distances between points
Spectral clustering	number of clusters	Medium n_samples, small n_clusters	Few clusters, even cluster size, non-flat geometry	Graph distance (e.g. nearest-neighbor graph)
Ward hierarchical clustering	number of clusters or distance threshold	Large n_samples and n_clusters	Many clusters, possibly connectivity constraints	Distances between points
Agglomerative clustering	number of clusters or distance threshold, linkage type, distance	Large n_samples and n_clusters	Many clusters, possibly connectivity constraints, non Euclidean distances	Any pairwise distance
DBSCAN	neighborhood size	Very large n_samples, medium n_clusters	Non-flat geometry, uneven cluster sizes	Distances between nearest points
OPTICS	minimum cluster membership	Very large n_samples, large n_clusters	Non-flat geometry, uneven cluster sizes, variable cluster density	Distances between points
Gaussian mixtures	many	Not scalable	Flat geometry, good for density estimation	Mahalanobis distances to centers
Birch	branching factor, threshold, optional global clusterer.	Large n_clusters and n_samples	Large dataset, outlier removal, data reduction.	Euclidean distance between points

基于划分(Partitioning)的聚类方法

- 给定N个对象，构造K个分组，每个分组就代表一个聚类。
- K个分组满足以下条件：
 - 每个分组至少包含一个对象；
 - 每个对象属于且仅属于一个分组；
- K-Means算法是最常见和典型的基于划分的聚类方法

K-Means聚类算法

- 数据类型与相似性的度量：

- 1) 连续属性

闵可夫斯基距离：

$$d(x_i, q_l) = \sqrt[p]{\sum_{j=1}^m |x_{ij} - q_{lj}|^p}$$

p 为正整数，p=1 时即为曼哈顿距离；p=2 时即为欧几里得距离。

- 2) 文档数据

对于文档数据使用余弦相似性度量，先将文档数据整理成文档—词矩阵格式：

	lost	win	team	score	music	happy	sad	...	coach
文档一	14	2	8	0	8	7	10	...	6
文档二	1	13	3	4	1	16	4	...	7
文档三	9	6	7	7	3	14	8	...	5

两个文档之间的相似度的计算公式为：

$$s(x_i, x_j) = \cos(x_i, x_j) = \frac{x_i \cdot x_j}{\|x_i\| \|x_j\|}$$

两个文档之间的距离的计算公式为：

$$d(x_i, x_j) = 1 - \cos(x_i, x_j)$$

K-Means聚类算法

- 目标函数

$$\min_{U,Z} \sum_{l=1}^k \sum_{i=1}^n u_{il} d^2(x_i, z_l) = \min_{U,Z} \|X - UZ\|_F^2$$

- $\sum_{l=1}^k u_{il} = 1 \quad 1 \leq i \leq n$
- $u_{il} \in \{0,1\} \quad 1 \leq i \leq n, 1 \leq l \leq k$

符号	含义
k	聚类簇的个数
z_l	第l个簇中心
X	对象（样本）
n	数据集中样本的个数
U	$n \times k$ 划分矩阵， $u_{il} = 1$ 表示第i个样本属于第l个簇
$d(.,.)$	距离，一般用欧式距离

两步求解法

- a) Fix $Z = \hat{Z}$ and solve the reduced problem

$$\min_U \sum_{l=1}^k \sum_{i=1}^n u_{il} d^2(x_i, \hat{z}_l) \quad \text{solution}$$

- $u_{il} = 1$ If $d^2(x_i, z_l) \leq d^2(x_i, z_t)$, for $1 \leq t \leq k$
- $u_{il} = 0$ for $t \neq l$

- b) Fix $U = \hat{U}$ and solve the reduced problem

$$\min_Z \sum_{l=1}^k \sum_{i=1}^n \hat{u}_{il} d^2(x_i, z_l) \quad \text{solution}$$

$$z_{il} = \frac{\sum_{i=1}^n u_{ij} x_{ij}}{\sum_{i=1}^n u_{ij}} \quad \text{for } 1 \leq l \leq k, 1 \leq j \leq m$$

k-means聚类算法

输入：待聚类的 n 个数据点，期望生成的聚类的个数 k

输出： k 个聚类

算法描述：

选出 K 个点作为初始的cluster center

Loop:

对输入中的每一个点 p :

{

计算 p 到各个cluster的距离;

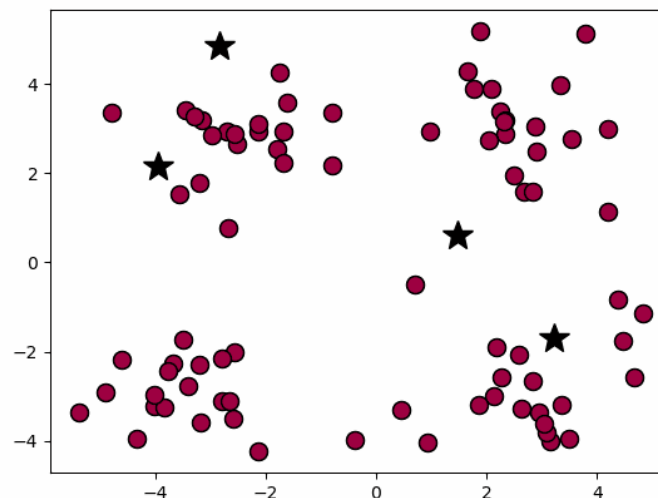
将 p 归入最近的cluster;

}

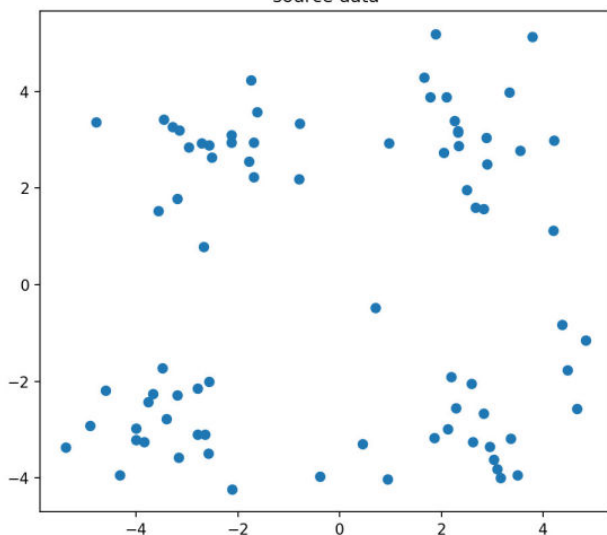
重新计算各个cluster的中心

如果不满足停止条件, goto Loop; 否则, 停止

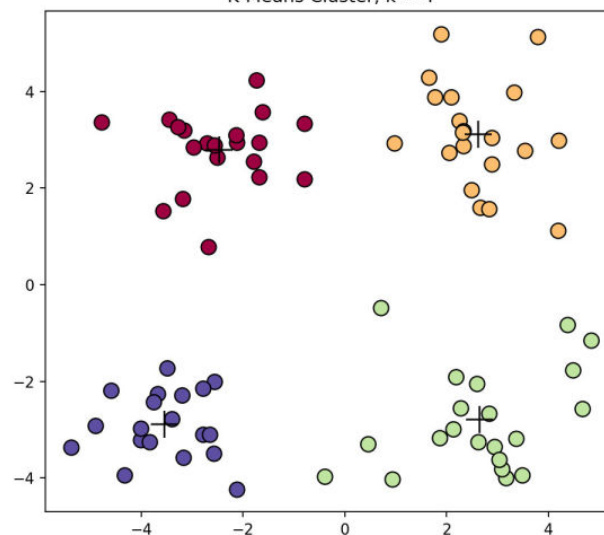
K-Means Cluster Process



source data



K-Means Cluster, $k = 4$



k-means聚类过程

过程示例

初始数据

$K = 2$



选择初始中心



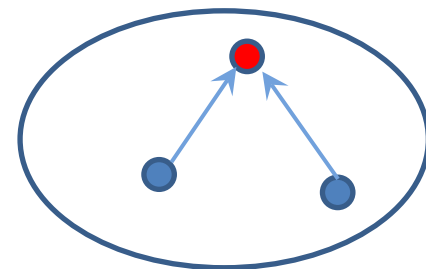
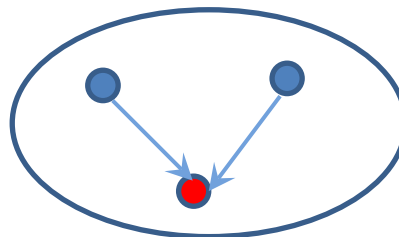
第1次聚类：计算距离



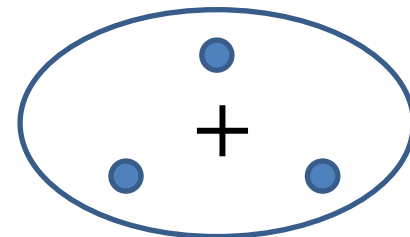
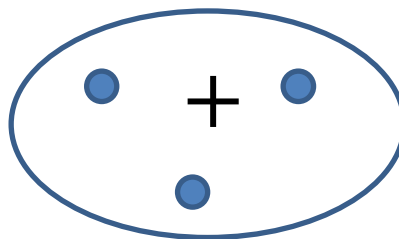
k-means聚类过程

过程示例(cont.)

第1次聚类：
归类各点



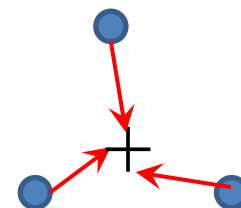
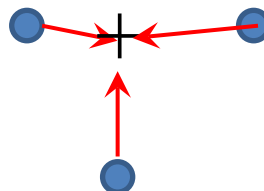
重新计算聚类中心



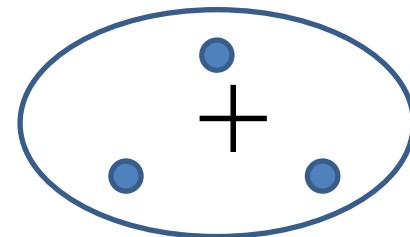
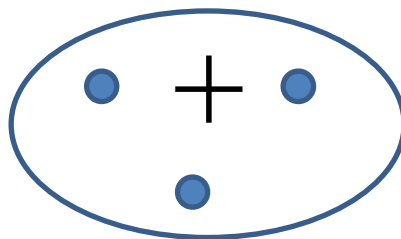
k-means聚类过程

过程示例(Cont.)

第2次聚类：
计算距离



第2次聚类：
归类各点



聚类无变化，迭代终止

Python k-means聚类参数

sklearn.cluster.KMeans

参数:

- **n_clusters:** 整形, 缺省值=8 [生成的聚类数, 即产生的质心 (centroids) 数]
- **max_iter:** 整形, 缺省值=300, 执行一次k-means算法所进行的最大迭代数
- **n_init:** 整形, 缺省值=10用不同的质心初始化值运行算法的次数, 最终解是在inertia意义下选出的最优结果。
- **init:** 有三个可选值: 'k-means++', 'random', 或者传递一个ndarray向量。此参数指定初始化方法, 默认值为 'k-means++'。(1) 'k-means++' 用一种特殊的方法选定初始质心从而能加速迭代过程的收敛(即上文中的k-means++介绍)(2) 'random' 随机从训练数据中选取初始质心。(3) 如果传递的是一个ndarray, 则应该形如 (n_clusters, n_features) 并给出初始质心。
- **precompute_distances:** 三个可选值, 'auto', True 或者 False。预计算距离, 计算速度更快但占用更多内存。(1) 'auto': 如果 样本数乘以聚类数大于 12million 的话则不预计算距离。This corresponds to about 100MB overhead per job using double precision.(2) True: 总是预先计算距离。(3) False: 永远不预先计算距离。
- **tol:** float形, 默认值=1e-4 与inertia结合来确定收敛条件。
- **n_jobs:** 整形数。指定计算所用的进程数。内部原理是同时进行n_init指定次数的计算。(1) 若值为 -1, 则用所有的CPU进行运算。若值为1, 则不进行并行运算, 这样的话方便调试。(2) 若值小于-1, 则用到的CPU数为(n_cpus + 1 + n_jobs)。因此如果 n_jobs值为-2, 则用到的CPU数为总CPU数减1。
- **random_state:** 整形或 numpy.RandomState 类型, 可选用于初始化质心的生成器(generator)。如果值为一个整数, 则确定一个seed。此参数默认值为numpy的随机数生成器。
- **copy_x:** 布尔型, 默认值=True。当我们precomputing distances时, 将数据中心化会得到更准确的结果。如果把此参数值设为True, 则原始数据不会被改变。如果是False, 则会直接在原始数据上做修改并在函数返回值时将其还原。但是在计算过程中由于有对数据均值的加减运算, 所以数据返回后, 原始数据和计算前可能会有细小差别。

Python k-means聚类参数

sklearn.cluster.KMeans

属性:

- `cluster_centers_`: 向量, $[n_clusters, n_features]$ (聚类中心的坐标)
- `Labels_`: 每个点的分组标签
- `inertia_`: float形, 每个点到其簇的质心的距离之和。

Notes:

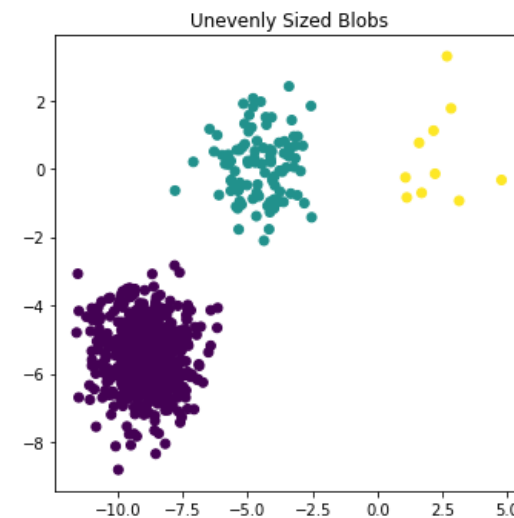
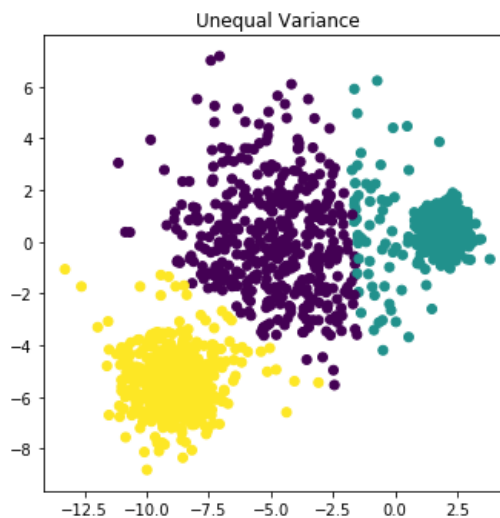
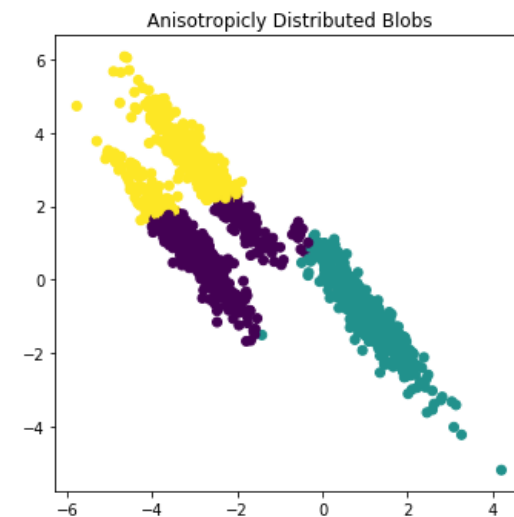
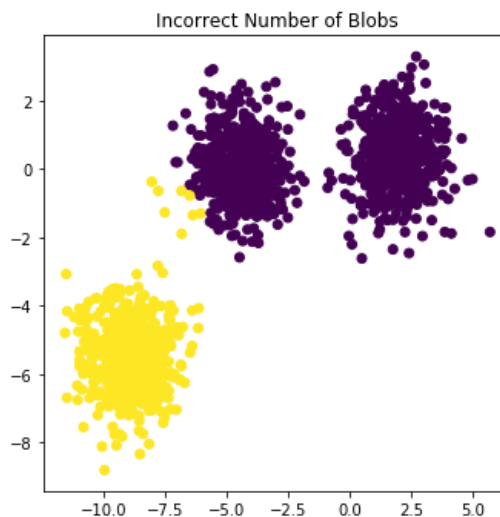
- 这个k-means运用了 Liloyd' s 算法, 平均计算复杂度是 $O(k*n*T)$, 其中 n 是样本量, T 是迭代次数。
- 计算复杂度在最坏的情况下为 $O(n^{(k+2/p)})$, 其中 n 是样本量, p 是特征个数。(D. Arthur and S. Vassilvitskii, 'How slow is the k-means method?' SoCG2006)
- 在实践中, k-means算法时非常快的, 属于可实践的算法中最快的那一类。但是它的解只是由特定初始值所产生的局部解。所以为了让结果更准确真实, 在实践中要用不同的初始值重复几次才可以。

Methods:

- `fit(X[,y])`: 计算k-means聚类。
- `fit_predict(X[,y])`: 计算簇质心并给每个样本预测类别。
- `fit_transform(X[,y])`: 计算簇并 transform X to cluster-distance space。
- `get_params([deep])`: 取得估计器的参数。
- `predict(X)`: 给每个样本估计最接近的簇。
- `score(X[,y])`: 计算聚类误差
- `set_params(**params)`: 为这个估计器手动设定参数。
- `transform(X[,y])`: 将X转换为簇距离空间。 在新空间中, 每个维度都是到集群中心的距离。 请注意, 即使X是稀疏的, 转换返回的数组通常也是密集的。

k-means聚类结果

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
plt.figure(figsize=(12, 12))
n_samples = 1500
random_state = 170
X, y = make_blobs(n_samples=n_samples, random_state=random_state)
# Incorrect number of clusters
y_pred = KMeans(n_clusters=2, random_state=random_state).fit_predict(X)
plt.subplot(221)
plt.scatter(X[:, 0], X[:, 1], c=y_pred)
plt.title("Incorrect Number of Blobs")
# Anisotropically distributed data
transformation = [[0.60834549, -0.63667341], [-0.40887718, 0.85253229]]
X_aniso = np.dot(X, transformation)
y_pred = KMeans(n_clusters=3, random_state=random_state).fit_predict(X_aniso)
plt.subplot(222)
plt.scatter(X_aniso[:, 0], X_aniso[:, 1], c=y_pred)
plt.title("Anisotropically Distributed Blobs")
# Different variance
X_varied, y_varied = make_blobs(n_samples=n_samples, cluster_std=[1.0, 2.5, 0.5], random_state=random_state)
y_pred = KMeans(n_clusters=3, random_state=random_state).fit_predict(X_varied)
plt.subplot(223)
plt.scatter(X_varied[:, 0], X_varied[:, 1], c=y_pred)
plt.title("Unequal Variance")
# Unevenly sized blobs
X_filtered = np.vstack((X[y == 0][:500], X[y == 1][:100], X[y == 2][:10]))
y_pred = KMeans(n_clusters=3, random_state=random_state).fit_predict(X_filtered)
plt.subplot(224)
plt.scatter(X_filtered[:, 0], X_filtered[:, 1], c=y_pred)
plt.title("Unevenly Sized Blobs")
plt.show()
```



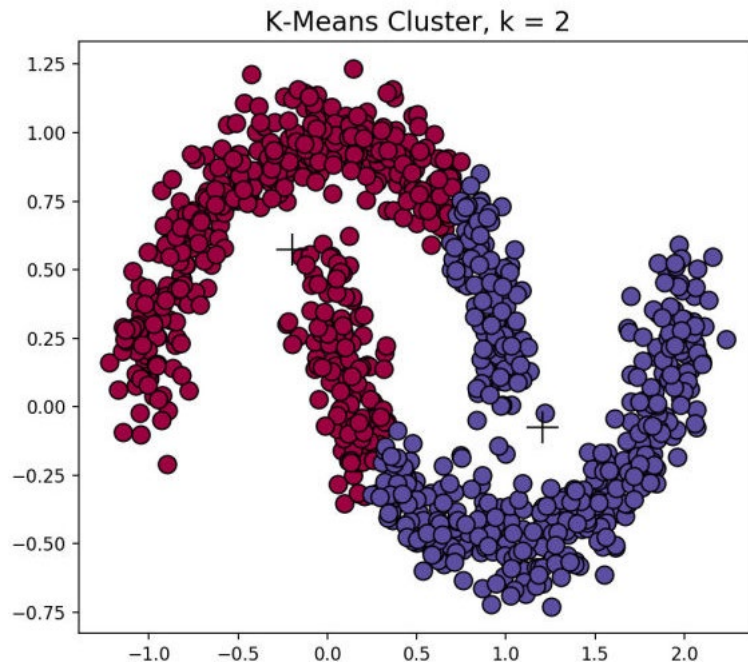
k-means聚类算法优缺点分析

优点：

1. 是解决聚类问题的一种经典算法，简单、快速
2. 对处理大数据集，该算法保持可伸缩性和高效性
3. 当簇接近高斯分布时，它的效果较好。

缺点：

1. 在簇的平均值可被定义的情况下才能使用，可能不适用于某些应用；
2. 在 K-means 算法中 K 是事先给定的，这个 K 值的选定是非常难以估计的。很多时候，事先并不知道给定的数据集应该分成多少个类别才最合适；
3. 在 K-means 算法中，首先需要根据初始聚类中心来确定一个初始划分，然后对初始划分进行优化。这个初始聚类中心的选择对聚类结果有较大的影响，一旦初始值选择的不好，可能无法得到有效的聚类结果；
4. 该算法需要不断地进行样本分类调整，不断地计算调整后的新的聚类中心，因此当数据量非常大时，算法的时间开销是非常大的；
5. 若簇中含有异常点，将导致均值偏离严重（即：对噪声和孤立点数据敏感）；
6. 不适用于发现非凸形状的簇或者大小差别很大的簇。



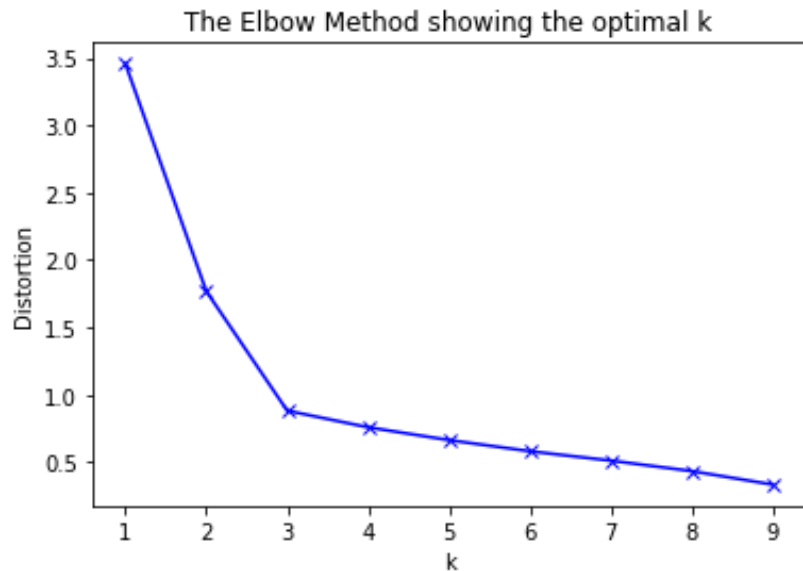
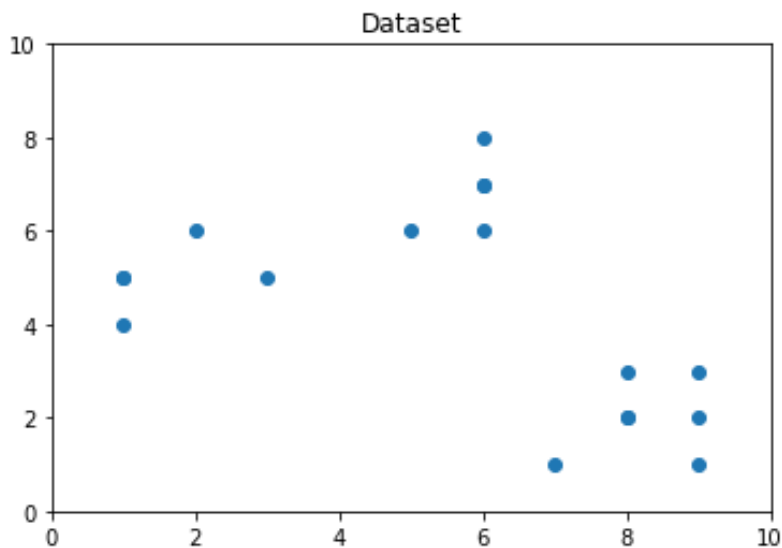
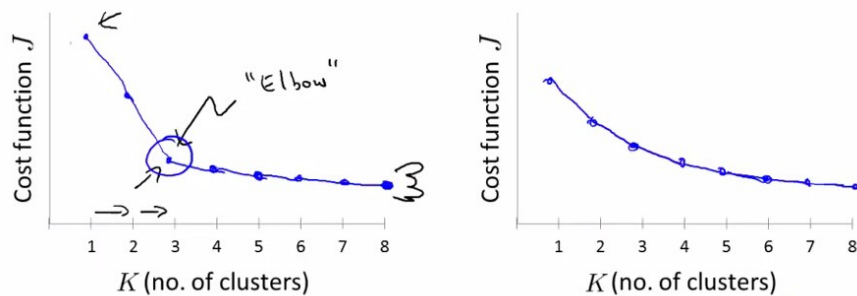
k值的选取

肘部法则 (Elbow method)

此种方法适用于 K 值相对较小的情况，当选择的 k 值小于真正的 K 时， k 每增加1， $cost$ 值就会大幅的减小；当选择的 k 值大于真正的 K 时， k 每增加1， $cost$ 值的变化就不会那么明显。这样，正确的 k 值就会在这个转折点，类似elbow的地方。

Choosing the value of K

Elbow method:



k值的选取

间隔统计量 Gap Statistic

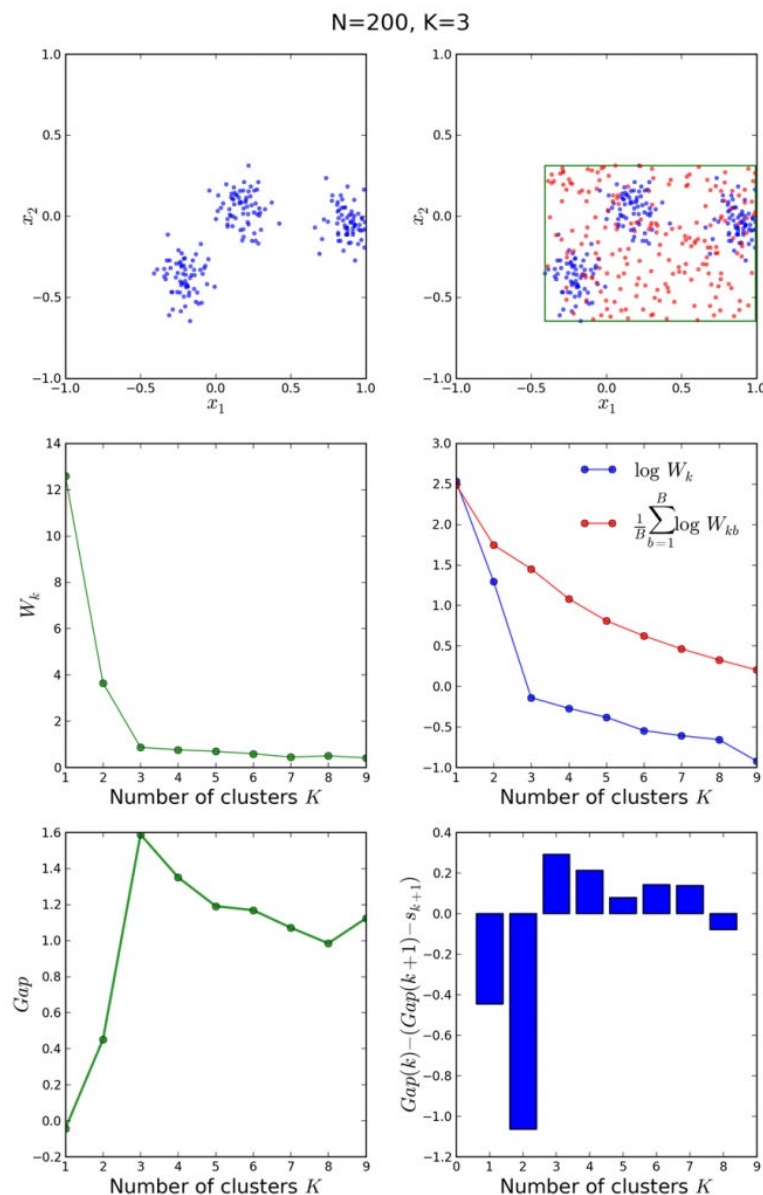
$$Gap_n(k) = E_n^*(\log W_k) - \log W_k$$

$$W_k = \sum_{l=1}^k \sum_{i=1}^n u_{il} d^2(x_i, z_l)$$

这里 $E(\log W_k)$ 指的是 $\log W_k$ 的期望，可以通过蒙特卡洛模拟产生：我们在样本里所在的矩形区域中（高维的话就是立方体区域）按照均匀分布随机地产生和原始样本数一样的随机样本，并对这个随机样本做K-Means，从而得到一个 W_k 。如此往复多次，通常20次，我们可以得到20个 $\log W_k$ 。对这20个数值求平均值，就得到了 $E(\log W_k)$ 的近似值。

选择满足 $Gap_n(k) \geq Gap_n(k+1) - s_{k+1}$ 的最小的k作为最优的聚类个数。

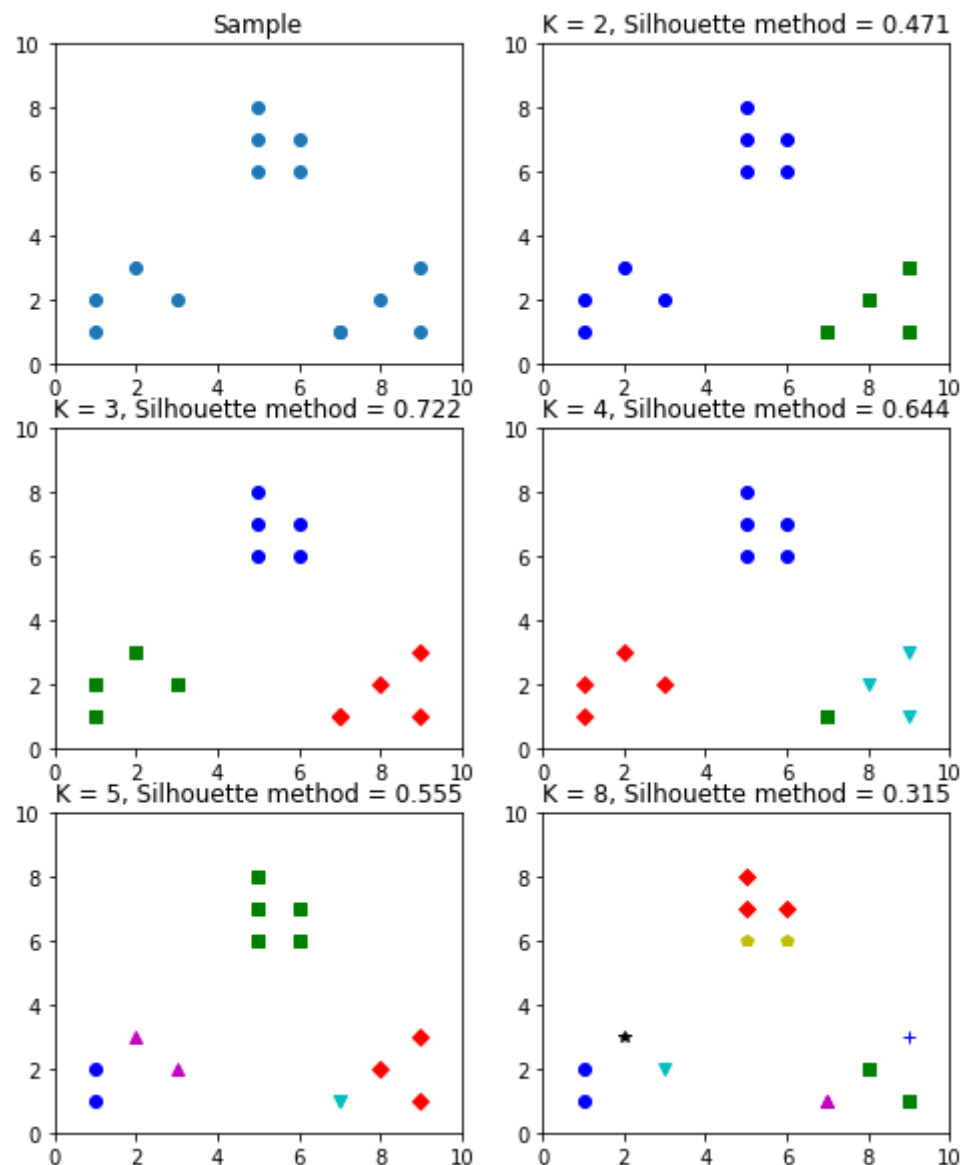
$$s_k = \sqrt{\frac{1+B}{B}} sd(k) \quad sd(k) = \sqrt{\frac{1}{B} \sum_{b=1}^B (\log W_{kb}^* - w')^2}$$
$$w' = \frac{1}{B} \sum_{b=1}^B \log W_{kb}^*$$



聚类分析——k值的选取

轮廓系数 (Silhouette Coefficient)

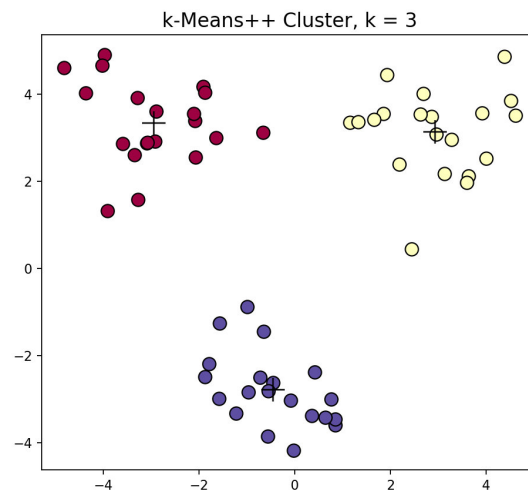
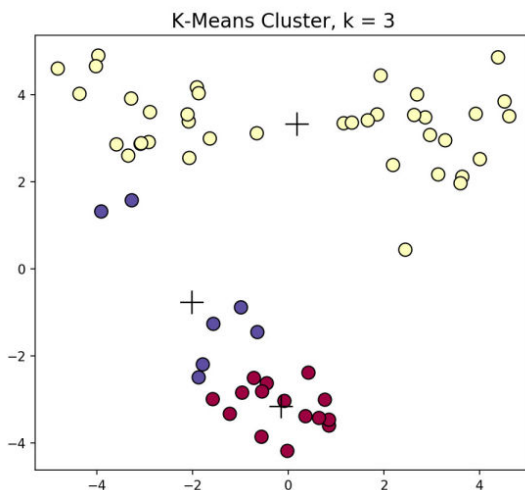
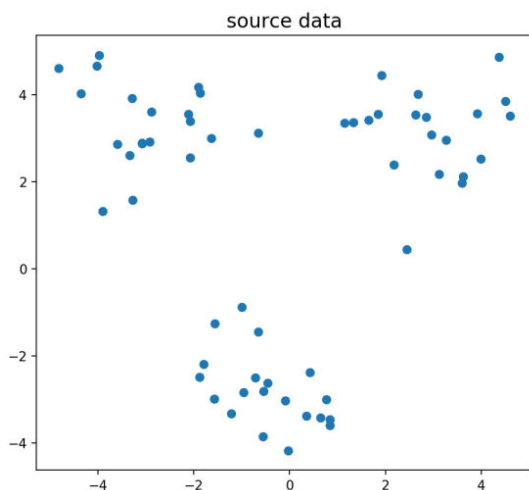
计算所有样本 $s(i)$ 的均值 s ，取使 s 最大的 k



聚类分析——k-means++

优化了初始中心选取方法：

- 从数据集中随机选取一个点作为初始聚类的中心 c_1
- 计算每个样本 x_i 与已有簇中心的最小距离，用 $d(x_i)$ 表示；然后计算每个样本被选为下一个聚类中心点的概率 $p(x_i) = \frac{d(x_i)}{\sum_{l=1}^n d^2(x_l)}$ ，之后通过轮盘法选出下一个聚类中心点
- 重复以上步骤直到选择出 k 个簇中心



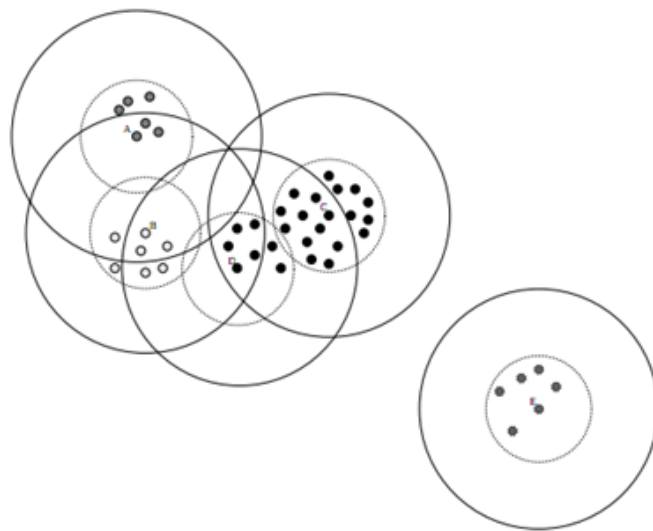
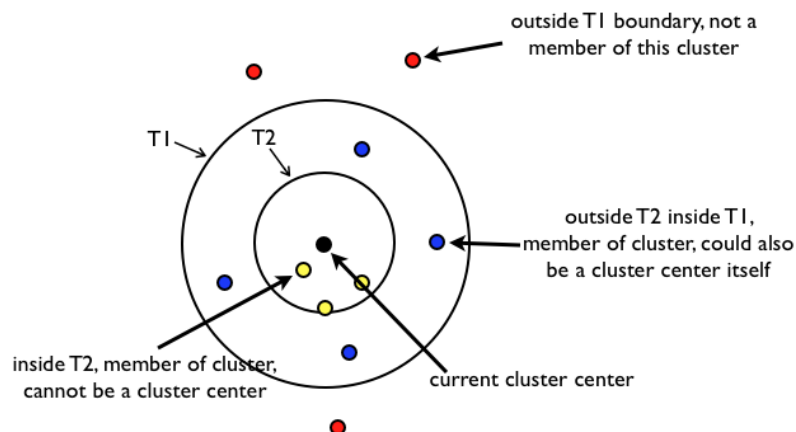
聚类分析——k-means+Canopy

Canopy算法的作用就在于它是通过事先粗聚类的方式，为k-means算法确定初始聚类中心个数和聚类中心点。

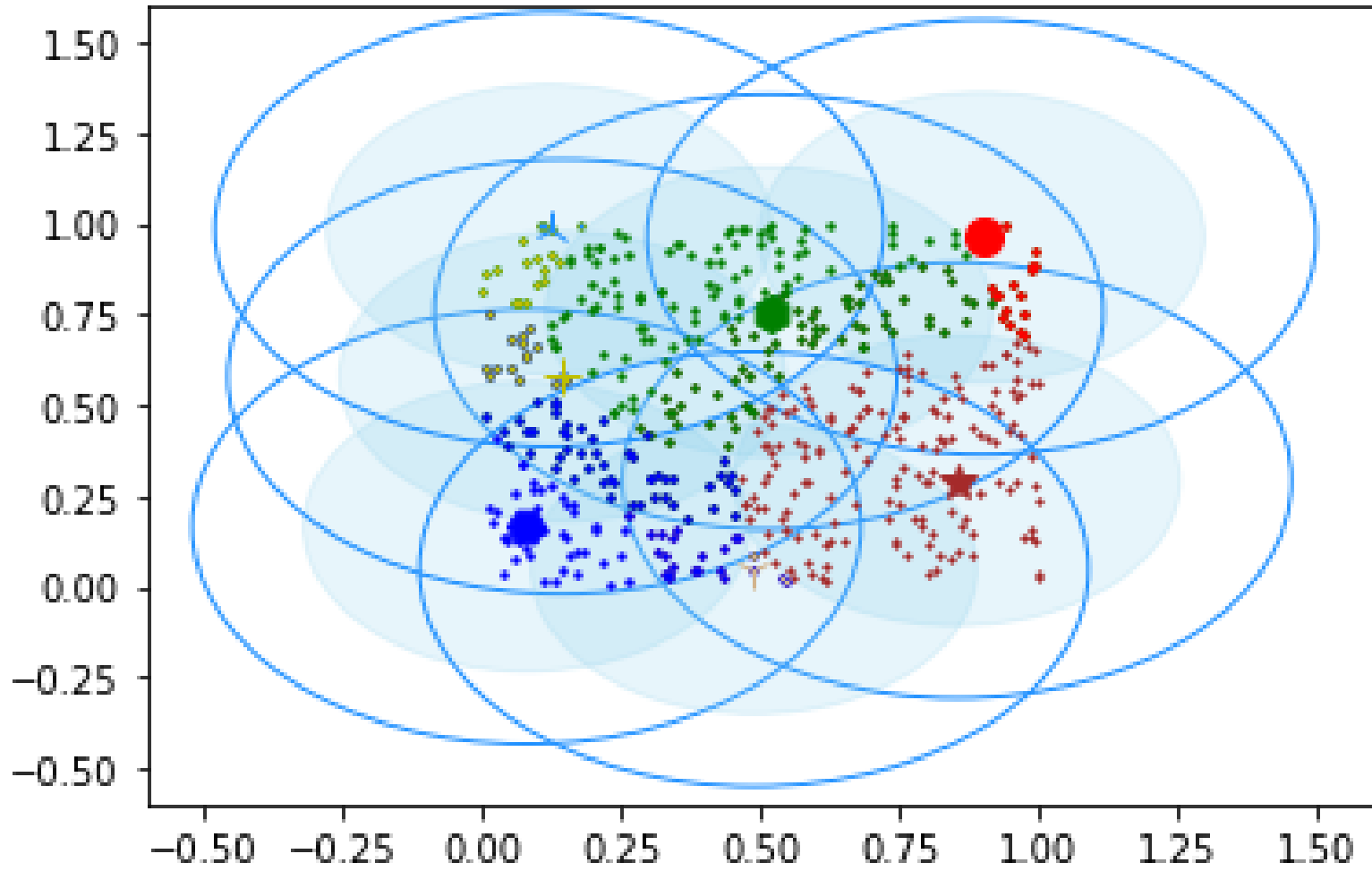
Canopy算法：

- 1.原始数据集**List**按照一定的规则进行排序（这个规则是任意的，但是一旦确定就不再更改），初始距离阈值为**T1**、**T2**，且**T1>T2**（**T1**、**T2**的设定可以根据用户的需要，或者使用交叉验证获得）。
- 2.在**List**中随机挑选一个数据向量**A**，使用一个粗糙距离计算方式计算**A**与**List**中其他样本数据向量之间的距离**d**。
- 3.根据第2步中的距离**d**，把**d**小于**T1**的样本数据向量划到一个**canopy**中，同时把**d**小于**T2**的样本数据向量从候选中心向量名单（这里可以理解为就是**List**）中移除。
- 4.重复第2、3步，直到候选中心向量名单为空，即**List**为空，算法结束。

算法原理比较简单，就是对数据进行不断遍历， $T2 < dis < T1$ 的可以作为中心名单， $dis < T2$ 的认为与**canopy**太近了，以后不会作为中心点，从**list**中删除，这样的话一个点可能属于多个**canopy**。



聚类分析——k-means+Canopy



Canopy聚类结果

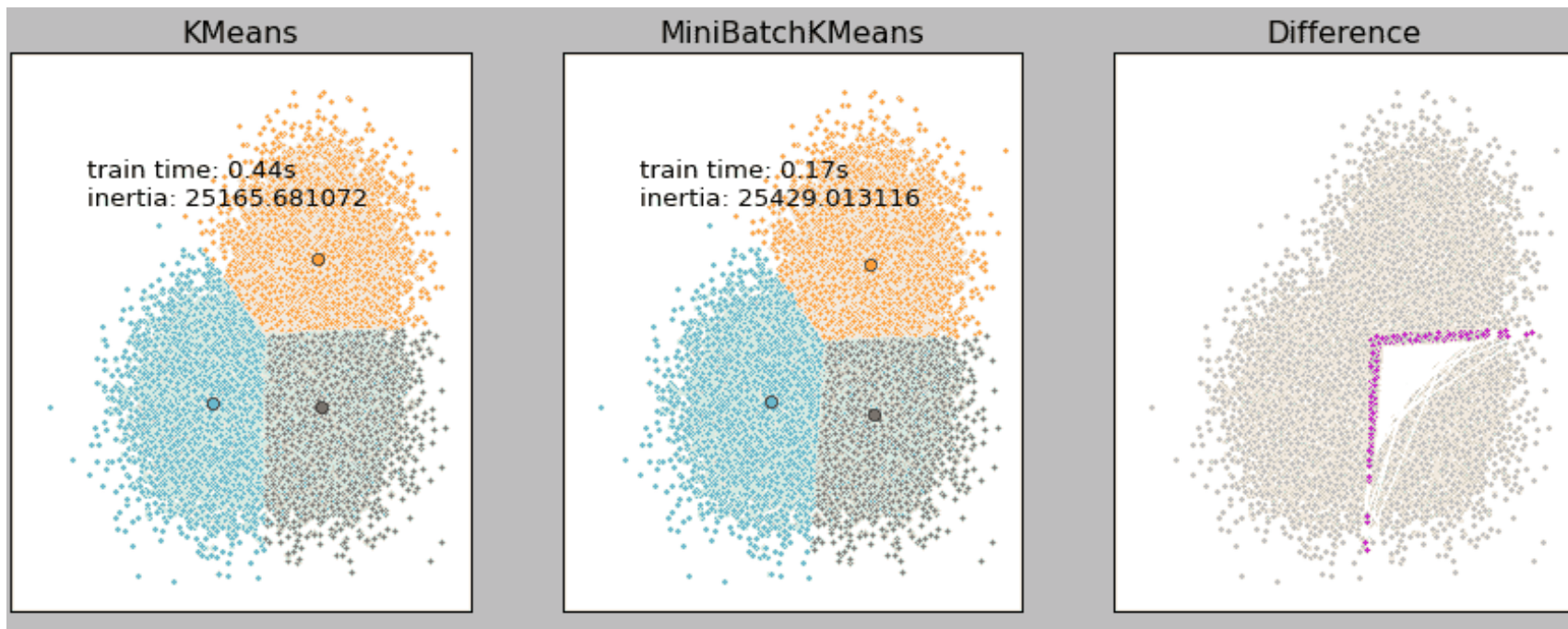
算法缺点：算法中 T_1 、 T_2 ($T_2 < T_1$) 的确定问题

聚类分析——二分k-means

- 将所有点作为一个簇，然后将该簇使用k-means划分为两个簇；
- 选择SSE（误差平方和）值最大的簇，使用k-means继续进行划分；
- 通过不断重复步骤二，直到达到需要的簇数量。

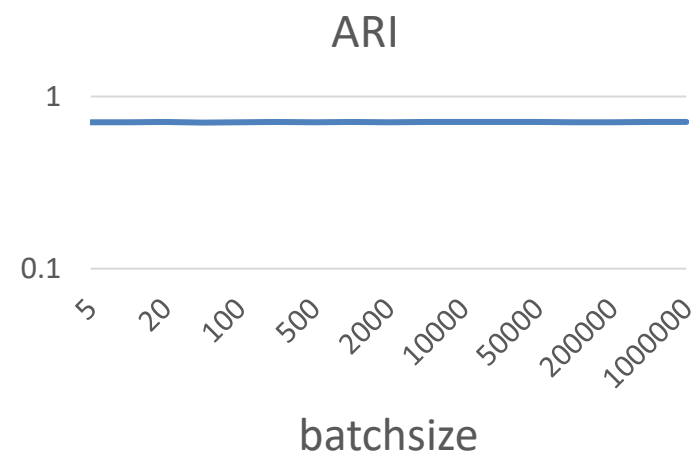
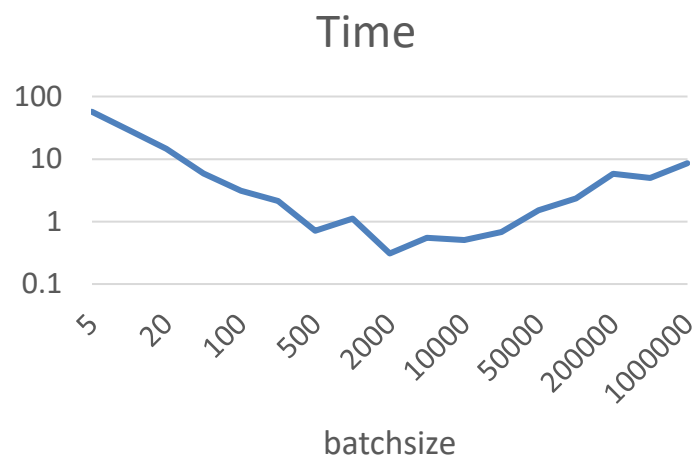
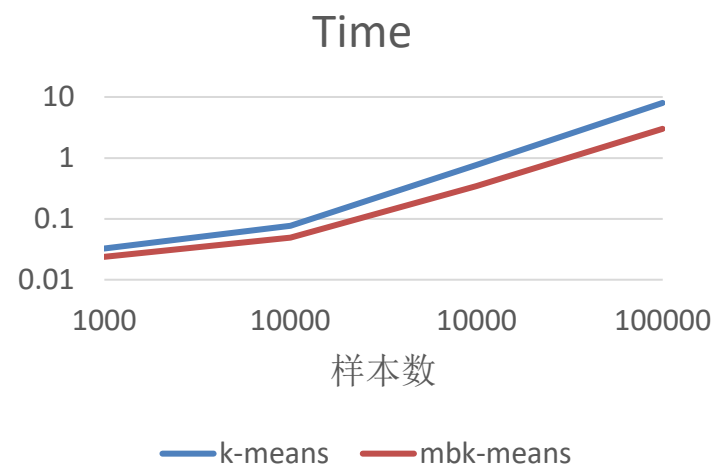
聚类分析——MiniBatchKMeans

Mini Batch K-Means算法是K-Means算法的变种，采用小批量的数据子集减小计算时间，同时仍试图优化目标函数。这里所谓的小批量是指每次训练算法时所随机抽取的数据子集，采用这些随机产生的子集进行训练算法，大大减小了计算时间，与其他算法相比，减少了k-均值的收敛时间，小批量k-均值产生的结果，一般只略差于标准算法。



3万的样本点分别使用K-Means和Mini Batch KMeans进行聚类的结果。由结果可知，在3万样本点的基础上，二者的运行时间相差2倍多，但聚类结果差异却很小（右侧粉红色的错误点）。

聚类分析——MiniBatchKMeans



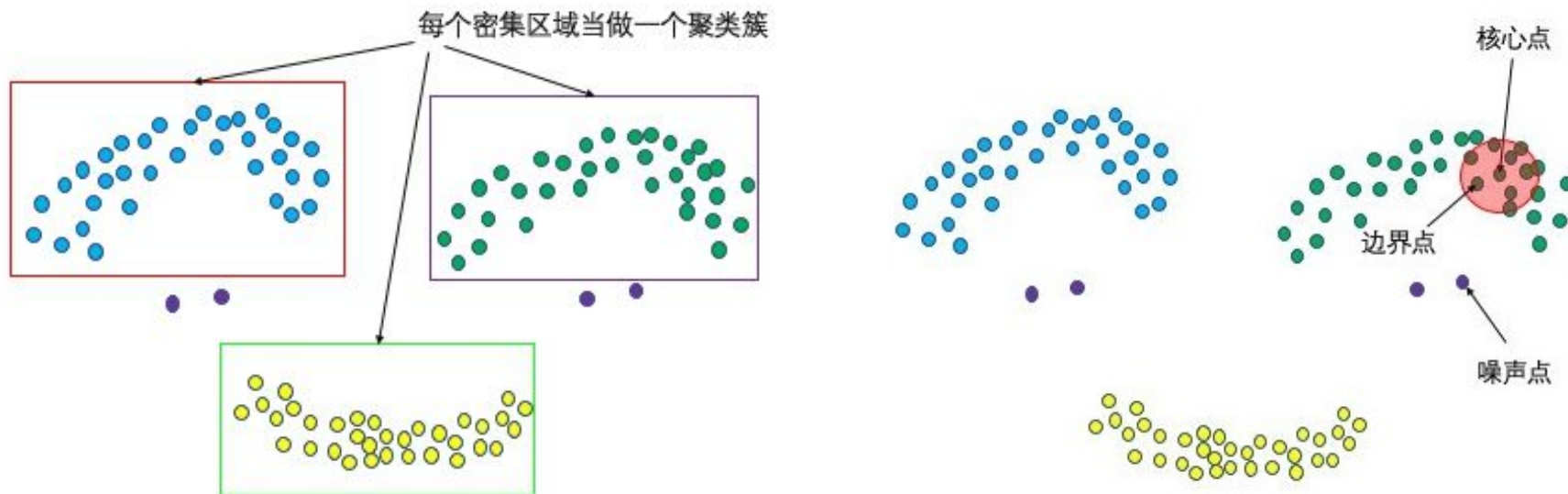
100万样本

100万样本

基于密度的聚类—DBSCAN聚类

Density-Based Spatial Clustering of Applications with Noise

1.DBSCAN 算法: 是一种基于密度的聚类算法，它使用一组关于“邻域”的参数来描述样本分布的紧密程度。DBSCAN 将簇定义为：由密度可达关系导出的最大的密度相连样本集合。

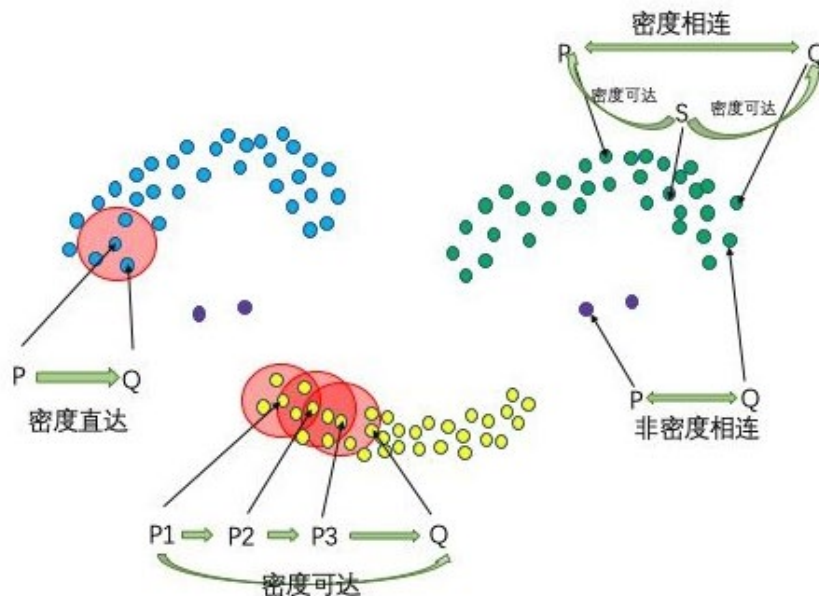


2.DBSCAN算法将数据点分为三类:

- 1.核心点: 在半径 ϵ ps内含有超过 $\min_pts$ 数目的点。
- 2.边界点: 在半径 ϵ ps内点的数量小于 $\min_pts$,但是落在核心点的邻域内的点。
- 3.噪音点: 既不是核心点也不是边界点的点。

基于密度的聚类—DBSCAN聚类中4种点的关系

- ◆ **密度直达**：如果P为核心点，Q在P的R邻域内，那么称P到Q密度直达。任何核心点到其自身密度直达，密度直达不具有对称性，如果P到Q密度直达，那么Q到P不一定密度直达。
- ◆ **密度可达**：如果存在核心点 $P_2, P_3, \dots, P_n$ ，且 P_1 到 P_2 密度直达， P_2 到 P_3 密度直达， $\dots$ ， P_{n-1} 到 P_n 密度直达， P_n 到Q密度直达，则 P_1 到Q密度可达。密度可达也不具有对称性。
- ◆ **密度相连**：如果存在核心点S，使得S到P和Q都密度可达，则P和Q密度相连。密度相连具有对称性，如果P和Q密度相连，那么Q和P也一定密度相连。密度相连的两个点属于同一个聚类簇。
- ◆ **非密度相连**：如果两个点不属于密度相连关系，则两个点非密度相连。非密度相连的两个点属于不同的聚类簇，或者其中存在噪声点。



基于密度的聚类—DBSCAN聚类算法流程

输入：样本集 $D=(x_1, x_2, \dots, x_m)$ ，邻域参数 $(\epsilon, \text{min_Pts})$ 样本距离度量方式

输出：簇划分 C .

1) 初始化核心对象集合 $\Omega=\emptyset$ ，初始化聚类簇数 $k=0$ ，初始化未访问样本集合 $\Gamma=D$ ，簇划分 $C=\emptyset$

2) 对于 $j=1, 2, \dots, m$ ，按下面的步骤找出所有的核心对象：

a) 通过距离度量方式，找到样本 x_j 的 ϵ -邻域子样本集 $N_\epsilon(x_j)$

b) 如果子样本集样本个数满足 $|N_\epsilon(x_j)| \geq \text{min_Pts}$ ，将样本 x_j 加入核心对象样本集合：

$\Omega = \Omega \cup \{x_j\}$

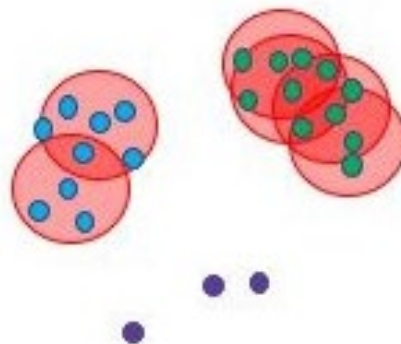
3) 如果核心对象集合 $\Omega=\emptyset$ ，则算法结束，否则转入步骤4.

4) 在核心对象集合 Ω 中，随机选择一个核心对象 o ，初始化当前簇核心对象队列 $\Omega_{\text{cur}}=\{o\}$ ，初始化类别序号 $k=k+1$ ，初始化当前簇样本集合 $C_k=\{o\}$ 更新未访问样本集合 $\Gamma = \Gamma - \{o\}$

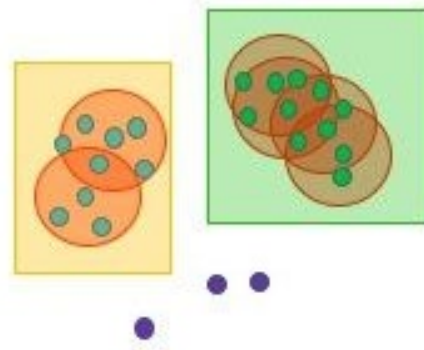
5) 如果当前簇核心对象队列 $\Omega_{\text{cur}}=\emptyset$ ，则当前聚类簇 C_k 生成完毕，更新簇划分 $C=\{C_1, C_2, \dots, C_k\}$ ，更新核心对象集合 $\Omega=\Omega-C_k$ ，转入步骤3。否则更新核心对象集合 $\Omega=\Omega-C_k$ 。

6) 在当前簇核心对象队列 Ω_{cur} 中取出一个核心对象 o' ，通过邻域距离阈值 ϵ 找出所有的 ϵ -邻域子样本集 $N_\epsilon(o')$ ，令 $\Delta=N_\epsilon(o') \cap \Gamma$ ，更新当前簇样本集合 $C_k=C_k \cup \Delta$ ，更新未访问样本集合 $\Gamma = \Gamma - \Delta$ ，更新 $\Omega_{\text{cur}}=\Omega_{\text{cur}} \cup (\Delta \cap \Omega) - o'$ ，转入步骤5.

输出结果为：簇划分 $C=\{C_1, C_2, \dots, C_k\}$

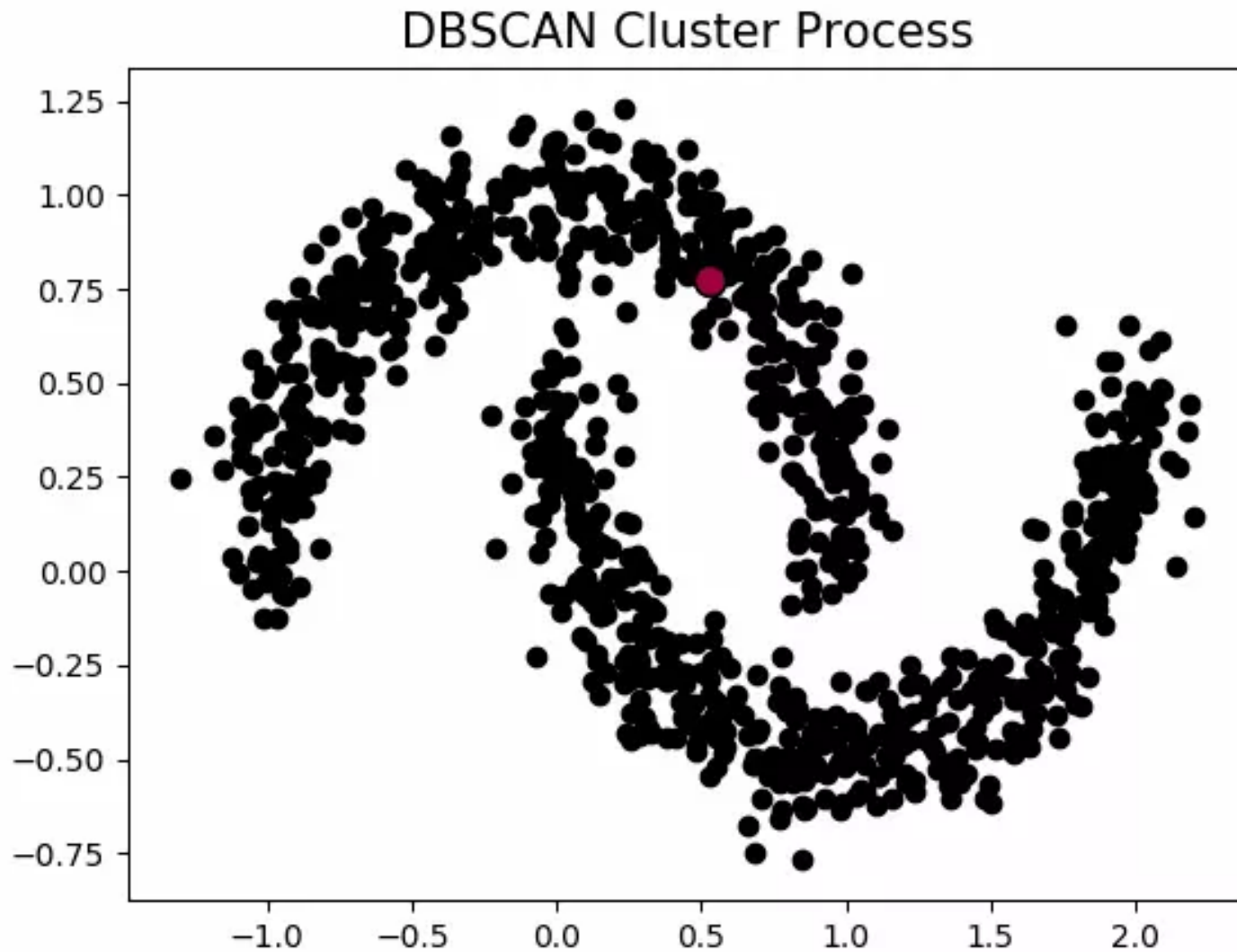


第一步，找到核心点形成临时聚类簇

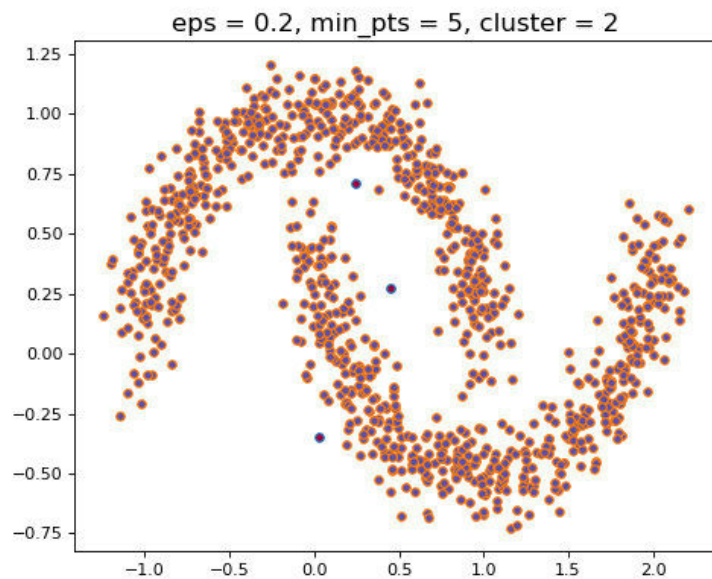
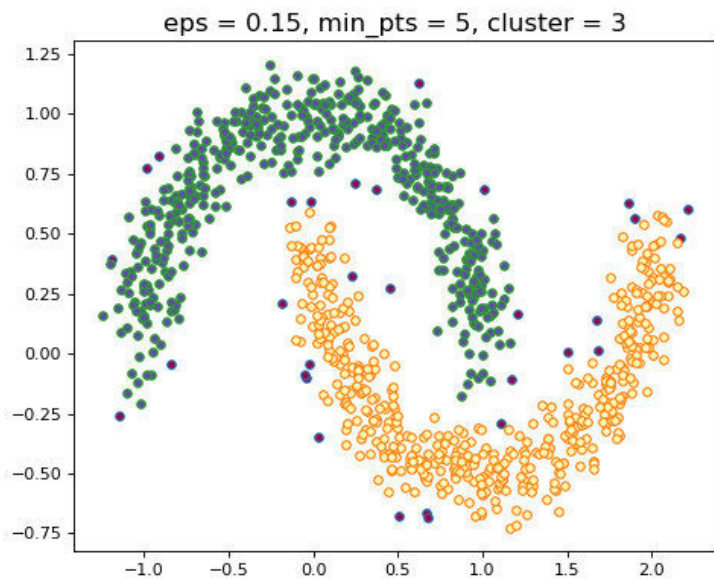
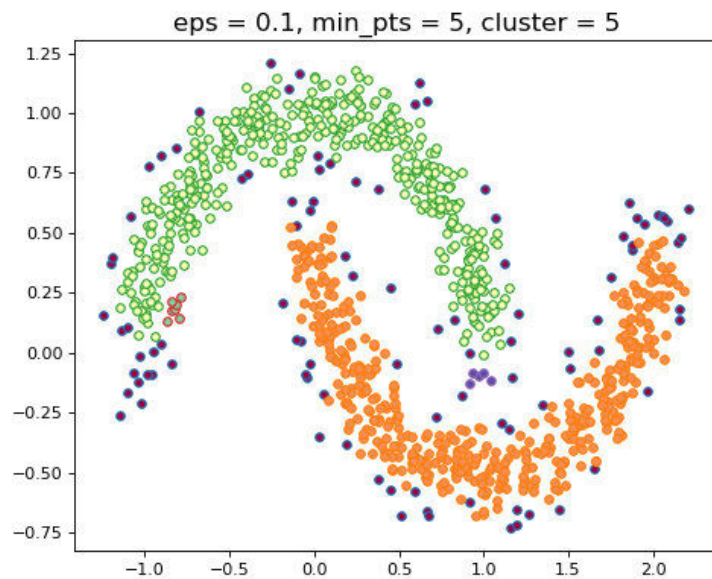
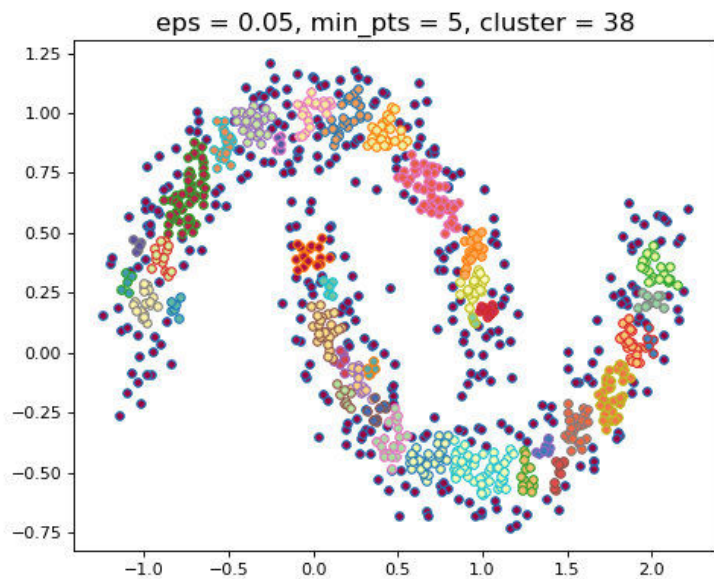


第二步，合并相连的临时聚类簇得到聚类簇

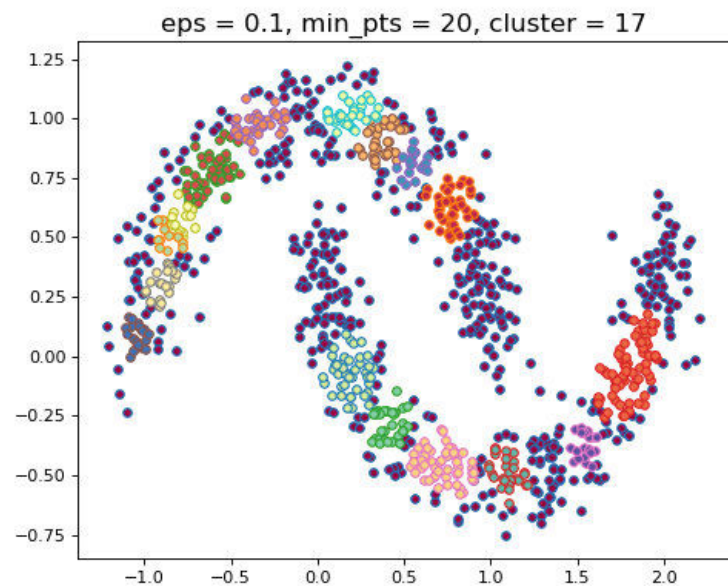
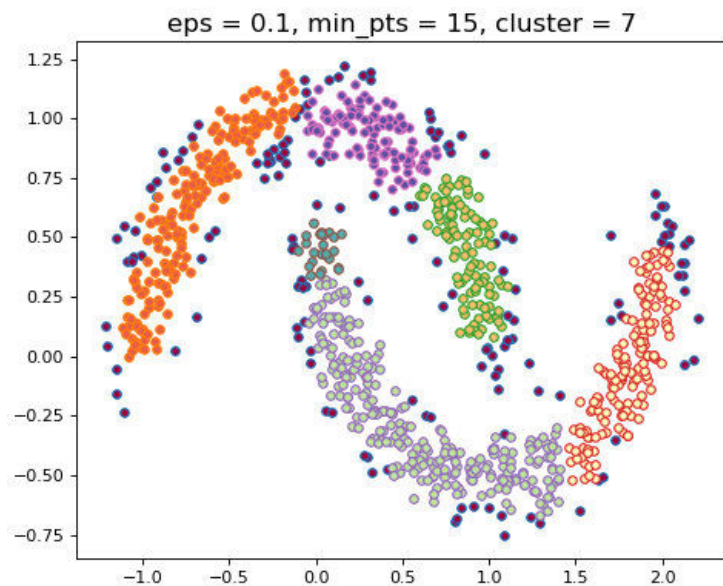
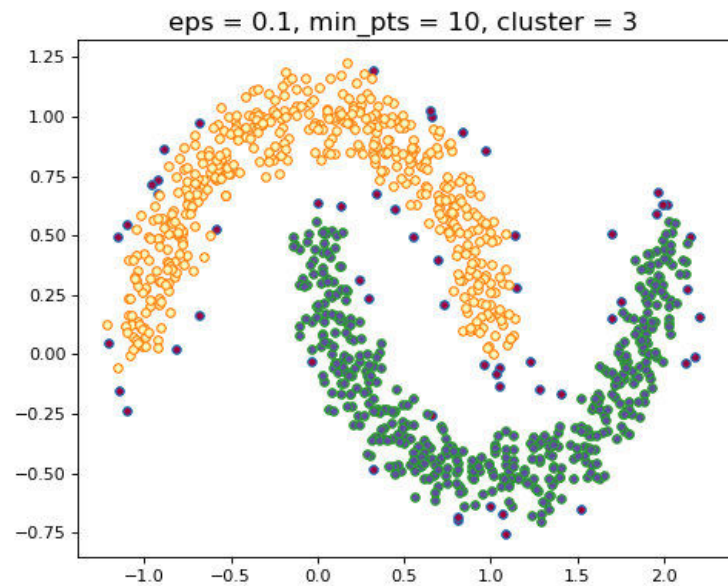
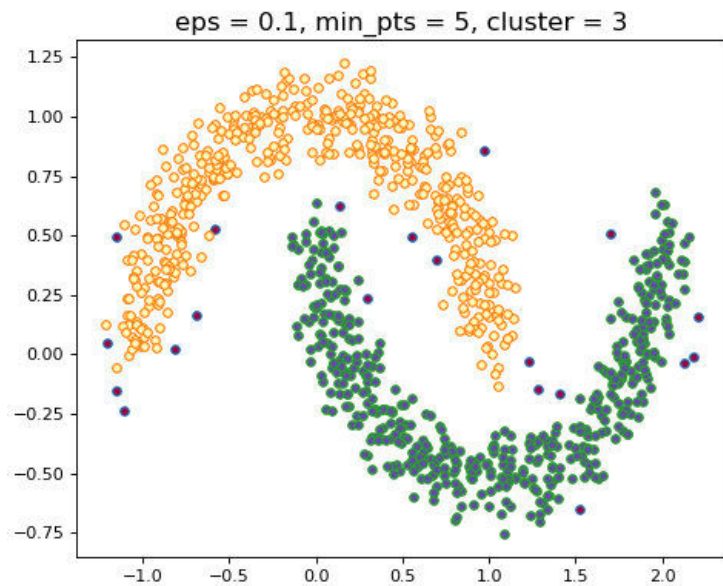
基于密度的聚类—DBSCAN聚类过程演示



基于密度的聚类—DBSCAN参数影响 ϵ



基于密度的聚类—DBSCAN参数影响min_pts



基于密度的聚类—DBSCAN聚类优缺点分析

优点：

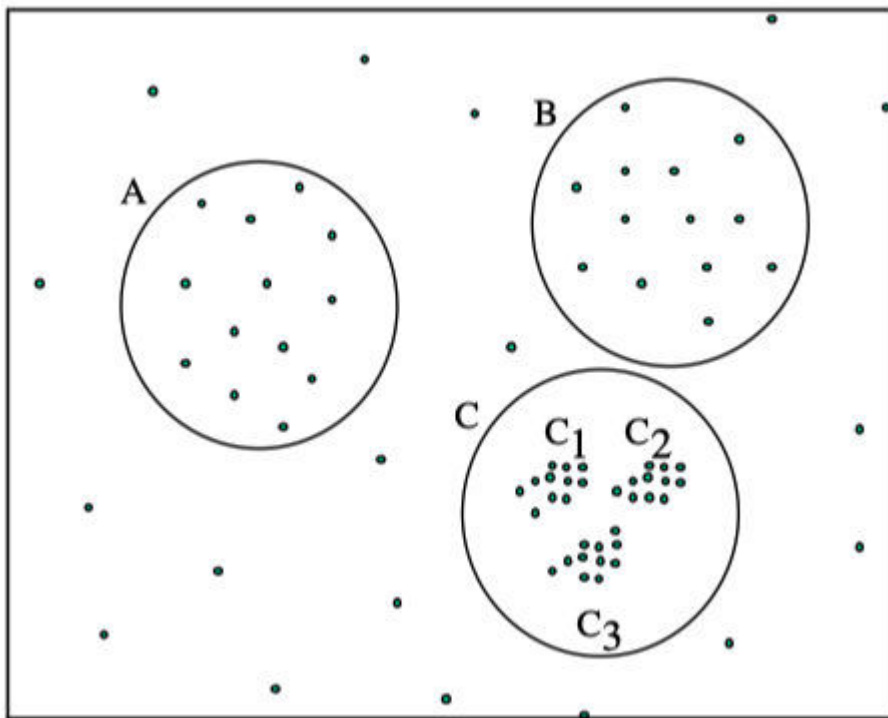
1. 不需要知道有多少个簇
2. 在聚类的同时发现异常点，对数据集中的异常点不敏感。
3. 对聚类的形状没有偏倚，可以找到任意大小和形状的簇。

缺点：

1. 需要为算法指定eps和MinPts参数，这对分析人员是一个很大的挑战
2. 对参数eps和MinPts的设置是非常敏感
3. 不适用于密度不均匀、聚类间距差相差很大的样本集

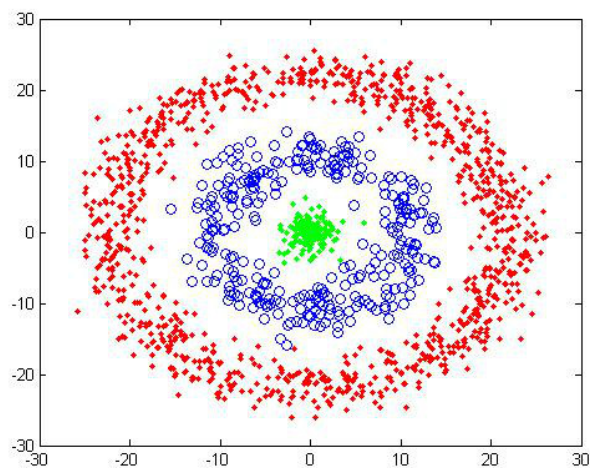
基于密度的聚类—DBSCAN的问题

在DBSCAN算法中，使用了统一的 ϵ 值，当数据密度不均匀的时候，如果设置了较小 ϵ 值，则较稀疏的cluster中的节点密度会小于 ϵ ，会被认为是边界点而不被用于进一步的扩展；如果设置了较大的 ϵ 值，则密度较大且离的比较近的cluster容易被划分为同一个cluster。

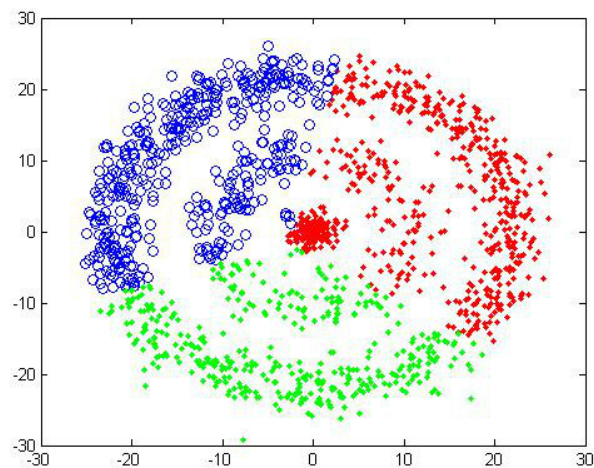


- 如果设置的 ϵ 较大，将会获得A, B, C这3个cluster
- 如果设置的 ϵ 较小，将会只获得C1、C2、C3这3个cluster

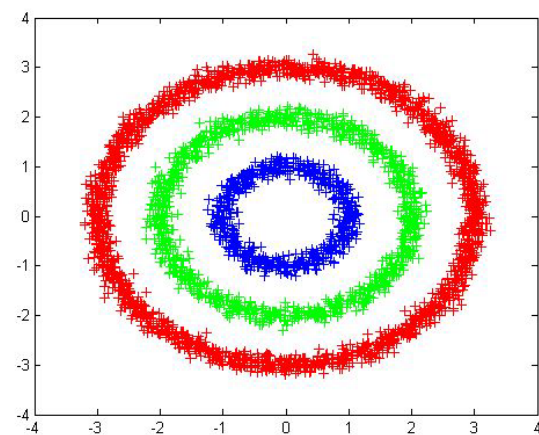
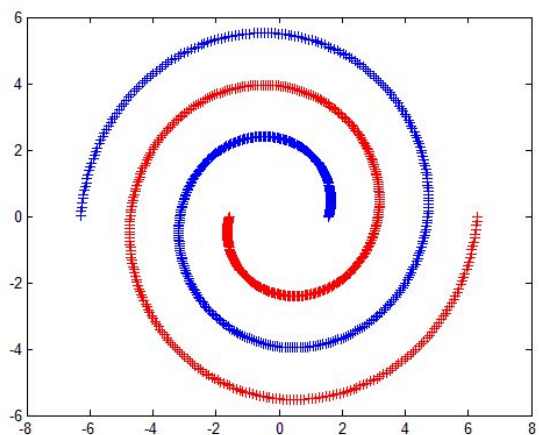
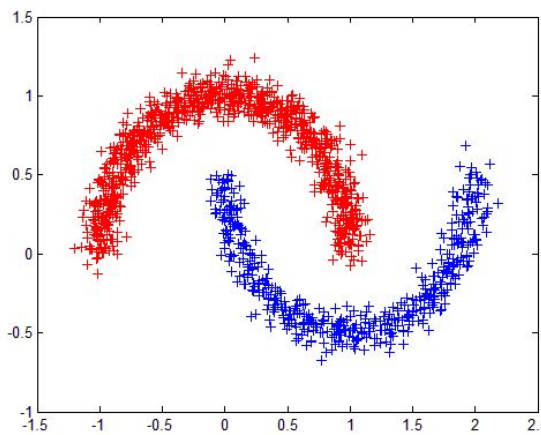
谱聚类——特点



Doughnut data



Clustering result by k-means



Clustering results by spectral clustering

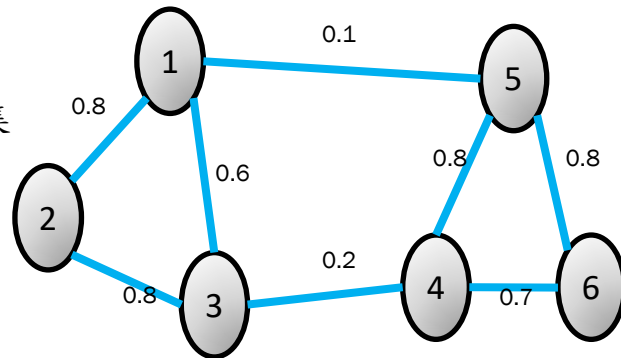
谱聚类——基本原理

谱聚类：是一种基于图论的聚类方法，通过对样本数据的拉普拉斯矩阵的特征向量进行聚类。

图（Graph）：由若干点及连接两点的线所构成的图形，通常用来描述某些事物之间的某种关系，用点代表事物，线表示对应两个事物间具有这种关系。

$G(V, E)$ 表示无向图， $V = \{v_1, v_2, \dots, v_n\}$ 表示点集， E 表示边集

w_{ij} 表示 v_i 与 v_j 之间的关系，称作权重，对于无向图 $w_{ij} = w_{ji}$ ，
而且 $w_{ii} = 0$ $w_{ij} \geq 0$



图划分（Graph partitioning）：指将图G完全划分为若干个（如k个）子图 $\{G_1, \dots, G_k\}$ ，各子图无交集

$$G_1 \cup \dots \cup G_k = G$$

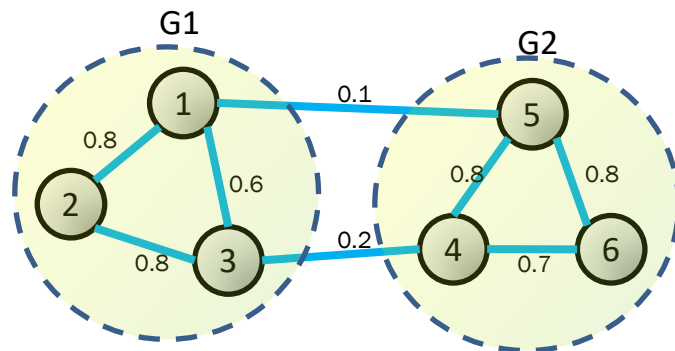
$$G_i \cap G_j = \emptyset$$

划分要求

同子图内的点相似度高
不同子图的点相似度低

最小划分时子图之间被“截断”的边的权重和

$$Cut(G_1, G_2) = \sum_{i \in G_1, j \in G_2} w_{ij}$$



谱聚类——权重矩阵

邻接矩阵 (W)：又称权重矩阵，是由任意两点之间的权重值 w_{ij} 组成的矩阵。

构建邻接矩阵的基本思想：

距离较远的两个点之间的边权重值较低，而距离较近的两个点之间的边权重值较高，可以通过样本点相似矩阵S 来构造邻接矩阵W。

(1) 全连接法：

通过核函数定义边权重，常用的有多项式核函数，高斯核函数和Sigmoid核函数。使用高斯核函数构建邻接矩阵：

$$w_{ij} = s_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

谱聚类——权重矩阵

(2) ε -近邻法:

设置一个距离阈值 ε ，然后计算任意两点 x_i 和 x_j 的相似度 s_{ij} ，然后根据 s_{ij} 与 ε 的关系来构建邻接矩阵 W ，如下：

$$w_{ij} = \begin{cases} 0 & s_{ij} < \varepsilon \\ s_{ij} & s_{ij} \geq \varepsilon \end{cases}$$

(3) K 近邻法:

利用KNN算法遍历所有的样本点，取每个样本最近的 K 个点作为近邻，只有和样本距离最近的 K 个点之间的 $w_{ij} > 0$ 。

这种方法会造成构建的权重矩阵 W 非对称，而后面的算法需要对称邻接矩阵，为了解决这个问题，一般采取下面两种方法之一。

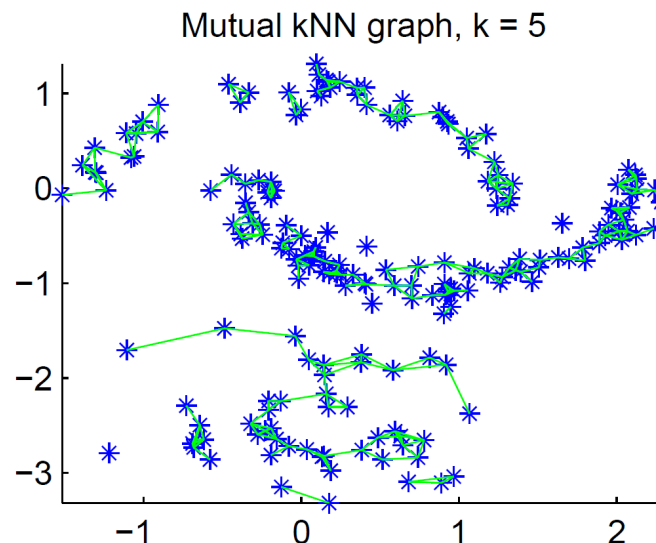
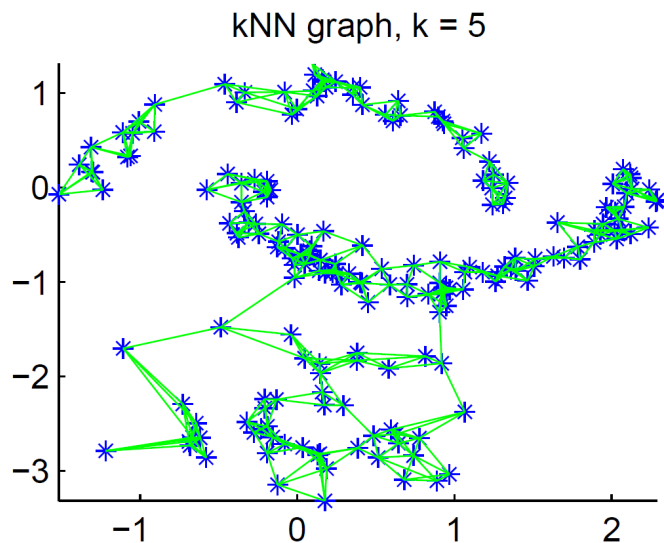
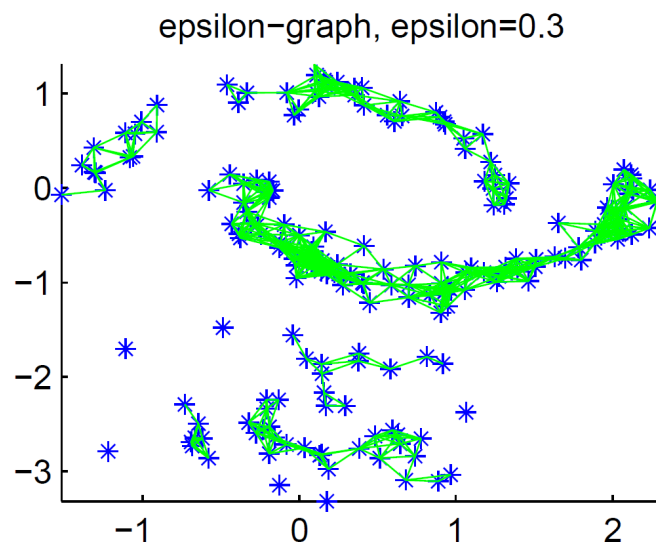
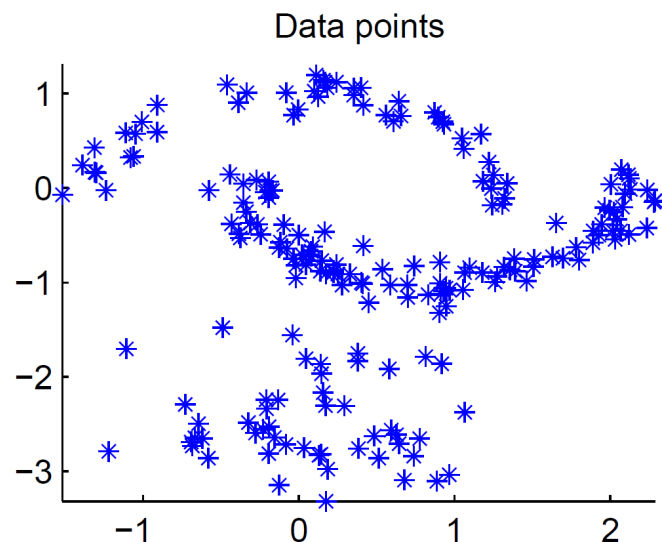
➤ 方法一：只要一个点在另一个点的 K 近邻中，则保留 s_{ij} 。

$$w_{ij} = w_{ji} = \begin{cases} 0 & x_i \notin KNN(x_j) \text{ and } x_j \notin KNN(x_i) \\ s_{ij} & x_i \in KNN(x_j) \text{ or } x_j \in KNN(x_i) \end{cases}$$

➤ 方法二：必须两个点互为 K 近邻时，才保留 s_{ij} 。

$$w_{ij} = w_{ji} = \begin{cases} 0 & x_i \notin KNN(x_j) \text{ or } x_j \notin KNN(x_i) \\ s_{ij} & x_i \in KNN(x_j) \text{ and } x_j \in KNN(x_i) \end{cases}$$

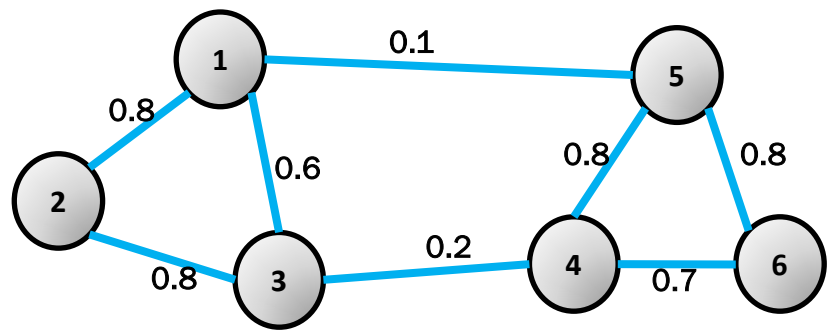
谱聚类——权重矩阵



使用不同的方法构造的权重矩阵

谱聚类——Laplacian矩阵

定义 $L = D - W$ ，称为拉普拉斯矩阵，W 为权重矩阵（邻接矩阵），D 为度矩阵（对角阵， $D_{ii} = \sum_{j=1}^n w_{ij}$ ）



邻接矩阵W

	1	2	3	4	5	6
1	0.0	0.8	0.6	0.0	0.1	0.0
2	0.8	0.0	0.8	0.0	0.0	0.0
3	0.6	0.8	0.0	0.2	0.0	0.0
4	0.0	0.0	0.2	0.0	0.8	0.7
5	0.1	0.0	0.0	0.8	0.0	0.8
6	0.0	0.0	0.0	0.7	0.8	0.0

度矩阵D

	1	2	3	4	5	6
1	1.5	0.0	0.0	0.0	0.0	0.0
2	0.0	1.6	0.0	0.0	0.0	0.0
3	0.0	0.0	1.6	0.0	0.0	0.0
4	0.0	0.0	0.0	1.7	0.0	0.0
5	0.0	0.0	0.0	0.0	1.7	0.0
6	0.0	0.0	0.0	0.0	0.0	1.5

Laplacian矩阵L=D-W

	1	2	3	4	5	6
1	1.5	-0.8	-0.6	0.0	-0.1	0.0
2	-0.8	1.6	-0.8	0.0	0.0	0.0
3	-0.6	-0.8	1.6	-0.2	0.0	0.0
4	0.0	0.0	-0.2	1.7	-0.8	-0.7
5	-0.1	0.0	0.0	-0.8	1.7	-0.8
6	0.0	0.0	0.0	-0.7	-0.8	1.5

谱聚类——Laplacian矩阵特性

L为半正定矩阵（即所有特征值非负值）：对任意一个n维向量 $q = [q_1, q_2, \dots, q_n]$ ，有

$$\begin{aligned} \sum_{i=1}^n \sum_{j=1}^n w_{ij} (q_i - q_j)^2 &= \sum_{i=1}^n \sum_{j=1}^n w_{ij} (q_i^2 - 2q_i q_j + q_j^2) \\ &= - \sum_{i=1}^n \sum_{j=1}^n 2w_{ij} q_i q_j + \sum_{i=1}^n \sum_{j=1}^n w_{ij} (q_i^2 + q_j^2) \\ &= - \sum_{i=1}^n \sum_{j=1}^n 2w_{ij} q_i q_j + \sum_{i=1}^n 2q_i^2 \sum_{j=1}^n w_{ij} \\ &= 2q^T Lq \end{aligned}$$

$$q^T Lq = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} (q_i - q_j)^2 \geq 0$$

L的最小特征值为0，且对应的特征向量为单位向量 $[1 \ 1 \ \dots \ 1]^T$

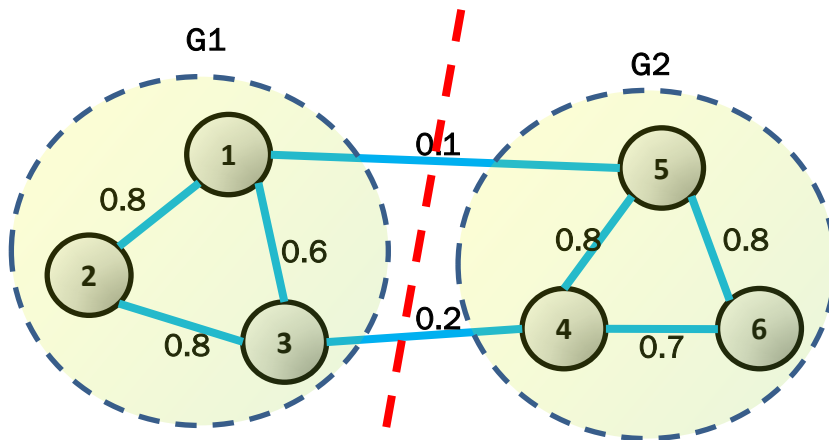
$$L[1 \ 1 \ \dots \ 1]^T = (D - W) * [1 \ 1 \ \dots \ 1]^T = [0 \ 0 \ \dots \ 0]^T = 0 * [1 \ 1 \ \dots \ 1]^T$$

谱聚类——Graph Cut

图划分是指将图完全划分为若干个子图，各子图无交集： $G_1 \cup \dots \cup G_k = G$ $G_i \cap G_j = \emptyset$

对于任意两个子图的集合 $A, B \subset G$, $A \cap B = \emptyset$, 定义A和B之间的切图权重为划分时子图之间被“截断”的边的权重和:

$$W(A, B) = \sum_{i \in A, j \in B} w_{ij} \quad \text{Cut}(G_1, \dots, G_k) = \frac{1}{2} \sum_{l=1}^k W(G_l, \overline{G_l}) = \frac{1}{2} \sum_{i \in G_l, j \in \overline{G_l}} w_{ij}$$



$$\text{Cut}(G_1, G_2) = \frac{1}{2} \sum_{i=1}^2 W(G_i, \overline{G_i}) = \sum_{i \in G_1, j \in G_2} w_{ij} = w_{15} + w_{34} = 0.3$$

三种方法

- ◆ Minimum Cut
- ◆ Ratio Cut
- ◆ Normalized Cut

谱聚类——Minimum Cut

对于k个子图的集合 $\{G_1, G_2, \dots, G_k\}$ ，定义切图Cut为：

$$Cut(G_1, G_2, \dots, G_k) = \sum_{l=1}^k cut(G_l, \overline{G_l})$$

其中 $\overline{G_l}$ 为 G_l 的补集。

Y 是簇指示矩阵，记为 $Y \in Ind$

$$y_{il} = \begin{cases} 1 & v_i \in G_l \\ 0 & v_i \notin G_l \end{cases}$$

$$\begin{aligned} y_l^T L y_l &= \frac{1}{2} \sum_{i,j=1}^n w_{ij} (y_{il} - y_{jl})^2 \\ &= \sum_{v_i \in G_l} \sum_{v_j \notin G_l} w_{ij} (y_{il} - y_{jl})^2 \\ &= \sum_{v_i \in G_l} \sum_{v_j \notin G_l} w_{ij} \\ &= cut(G_l, \overline{G_l}) \end{aligned}$$

$$y_l^T L y_l = (Y^T L Y)_{ll}$$

$$\begin{aligned} MinCut(G_1, \dots, G_k) &= \sum_{l=1}^k y_l^T L y_l \\ &= \sum_{l=1}^k (Y^T L Y)_{ll} \\ &= Tr(Y^T L Y) \end{aligned}$$

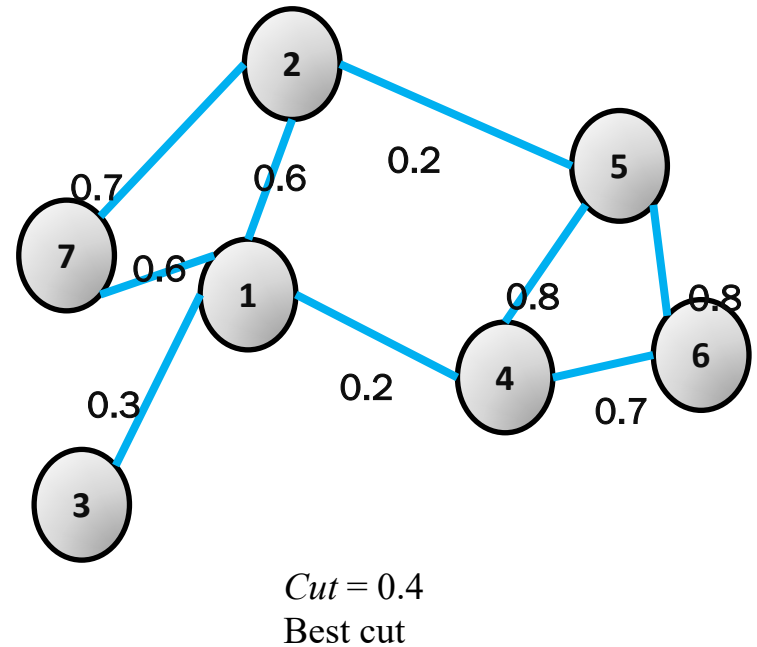
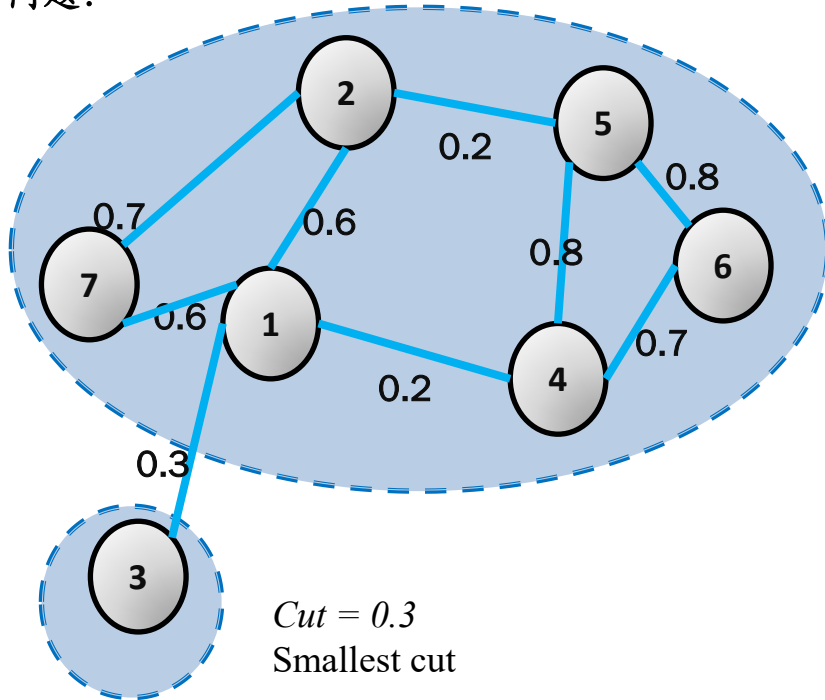
$Tr(\cdot)$ 为迹运算

图的划分问题转化为 $Tr(Y^T L Y)$ 最小值问题

求解： $\min_{Y \in Ind} Tr(Y^T L Y)$

谱聚类——Minimum Cut

问题:



Minimum Cut划分不均衡

如何解决? 一个自然的想法就是, 类似为了防止过拟合而添加正则项一样, 可以添加新的限定条件。
Ratio cut和Normalized cut

谱聚类——Ratio Cut

$$Rcut(G_1, \dots, G_k) = \sum_{l=1}^k \frac{Cut(G_l, \bar{G}_l)}{|G_l|} = \sum_{l=1}^k \frac{y_l^T L y_l}{y_l^T y_l}$$

避免生成太小的簇

引入簇指示向量 $Y \in \mathbb{R}^{n \times k}$

$$y_{il} = \begin{cases} \sqrt{\frac{1}{|G_l|}} & v_i \in G_l \\ 0 & v_i \notin G_l \end{cases} \quad Y^T Y = I$$

$$\begin{aligned} y_l^T L y_l &= \frac{1}{2} \sum_{i,j=1}^n w_{ij} (y_{il} - y_{jl})^2 \\ &= \sum_{v_i \in G_l} \sum_{v_j \notin G_l} w_{ij} (y_{il} - y_{jl})^2 \\ &= \sum_{v_i \in G_l} \sum_{v_j \notin G_l} \frac{w_{ij}}{|G_l|} \\ &= \frac{Cut(G_l, \bar{G}_l)}{|G_l|} \end{aligned}$$

$$y_l^T L y_l = (Y^T L Y)_{ll}$$

$$\begin{aligned} Rcut(G_1, \dots, G_k) &= \sum_{l=1}^k y_l^T L y_l \\ &= \sum_{l=1}^k (Y^T L Y)_{ll} \\ &= Tr(Y^T L Y) \end{aligned}$$

Minimum Cut重写为

松弛Y为连续矩阵

$$\min_{Y \in \mathbb{R}^{n \times k}} Tr(Y^T L Y) \quad \longrightarrow \quad \min_{Y \in \mathbb{R}^{n \times k}, Y^T Y = I} Tr(Y^T L Y)$$

根据Rayleigh-Ritz定理，以上问题的最优解为L最小的k个特征值所对应的k个特征向量组成的矩阵

谱聚类——Normalized Cut

$$Ncut(G_1, \dots, G_k) = \sum_{l=1}^k \frac{Cut(G_l, \bar{G}_l)}{\text{vol}(G_l)} = \sum_{l=1}^k \frac{y_l^T L y_l}{y_l^T D y_l}$$

$$\text{vol}(G_l) = \sum_{v_i \in G_l} d_{ii} \quad \text{避免生成太小的簇}$$

引入簇指示向量 $Y \in \mathbb{R}^{n \times k}$

$$y_{il} = \begin{cases} \sqrt{\frac{1}{\text{vol}(G_l)}} & v_i \in G_l \\ 0 & v_i \notin G_l \end{cases} \quad Y^T D Y = I$$

$$\begin{aligned} y_l^T L y_l &= \frac{1}{2} \sum_{i,j=1}^n w_{ij} (y_{il} - y_{jl})^2 \\ &= \sum_{v_i \in G_l} \sum_{v_j \notin G_l} w_{ij} (y_{il} - y_{jl})^2 \\ &= \sum_{v_i \in G_l} \sum_{v_j \notin G_l} \frac{w_{ij}}{\text{vol}(G_l)} \\ &= \frac{Cut(G_l, \bar{G}_l)}{\text{vol}(G_l)} \end{aligned}$$

$$\begin{aligned} y_l^T L y_l &= (Y^T L Y)_{ll} \\ Ncut(G_1, \dots, G_k) &= \sum_{l=1}^k y_l^T L y_l \\ &= \sum_{l=1}^k (Y^T L Y)_{ll} \\ &= \text{Tr}(Y^T L Y) \end{aligned}$$

松弛Y，将Minimum $cut(G_1, \dots, G_k)$ 重写为 $L_{norm} = D^{-1/2} L D^{-1/2}$

$$\boxed{\min_{Y \in \mathbb{R}^{n \times k}, Y^T D Y = I} \text{Tr}(Y^T L Y)} \xrightarrow[\substack{H = D^{1/2} Y \\ Y = D^{-1/2} H}]{\substack{H = D^{1/2} Y \\ Y = D^{-1/2} H}} \boxed{\min_{H \in \mathbb{R}^{n \times k}, H^T H = I} \text{Tr}(H^T L_{norm} H)}$$

根据Rayleigh-Ritz定理及[1]，以上问题的最优解H为 L_{norm} 最小的k个特征值所对应的k个特征向量组成的矩阵

$$D^{-1/2} L D^{-1/2} H = \lambda H \Leftrightarrow D^{-1/2} L D^{-1/2} D^{1/2} Y = \lambda D^{1/2} Y \Leftrightarrow D^{-1} L Y = \lambda Y$$

所以Y为 $D^{-1} L$ 最小的k个特征值所对应的k个特征向量组成的矩阵

1. Shi, J. and Malik, J. (2000). Normalized cuts and image segmentation. IEEE Transactions on Pattern Analysis and Machine Intelligence, 22 (8), 888 – 905.
2. Ulrike von Luxburg. A Tutorial on Spectral Clustering.
3. Lütkepohl, H. (1997). Handbook of Matrices. Chichester: Wiley.

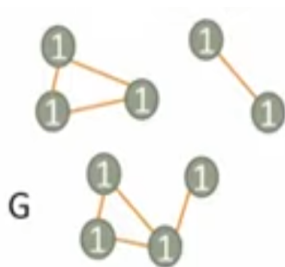
谱聚类——Unnormalized Spectral Clustering步骤

➤ Unnormalized Spectral Clustering

- Ratio Cut
- Minimum Cut

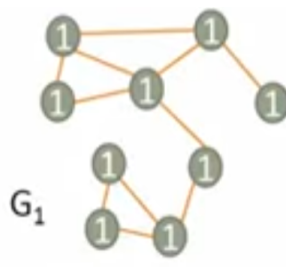
输入：样本及类别数K

- 1、根据样本建立权重矩阵 W ；
- 2、根据 W ，计算度矩阵 D ，进而计算拉普拉斯矩阵 L ；
- 3、计算 L 的特征值及特征向量 $V_e = \{v_{e1}, \dots, v_{en}\}$ ；
- 4、取出前K小特征值对应的特征向量 $V_K = \{v_{e1}, v_{e2}, \dots, v_{ek}\}$ 并对矩阵 V_K 的行向量进行聚类，得到K个Clusters



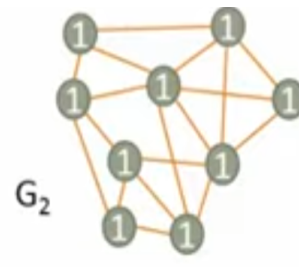
L_G has $0 = \lambda_1 = \lambda_2 = \lambda_3$ and $\lambda_4 > 0$

前三个特征值为0



Both L_{G1} and L_{G2} have $0 = \lambda_1$ and $\lambda_2 > 0$. $\lambda_2(L_{G1}) < \lambda_2(L_{G2})$

最小特征值为0，第二小特征值大于0；图2特征值更大，更连通



谱聚类——Normalized Spectral Clustering步骤

Normalized Cut (NCut)

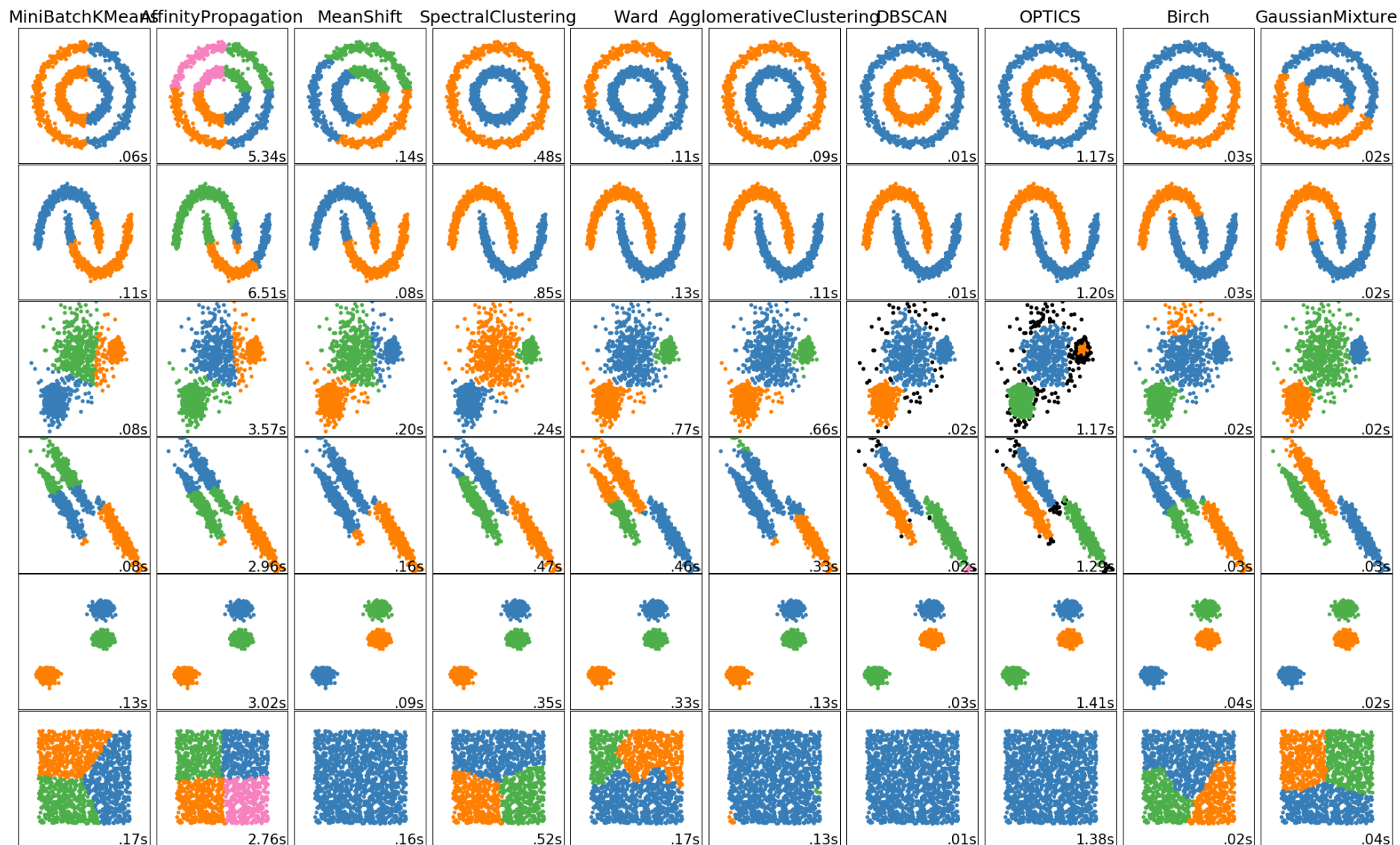
输入：样本及类别数k

- 1、根据样本建立权重矩阵 W ；
- 2、根据 W ，计算度矩阵 D ，进而计算拉普拉斯矩阵 $L_{norm} = D^{1/2} L D^{1/2}$ ；
- 3、计算 L_{norm} 的特征值及特征向量 $V_e = \{v_{e1}, \dots, v_{en}\}$ ；
- 4、取出前K小特征值对应的特征向量 $V_K = \{v_{e1}, v_{e2}, \dots, v_{ek}\}$ 并对矩阵 V_K 的行向量进行聚类，得到K个Clusters

谱聚类——算法意义

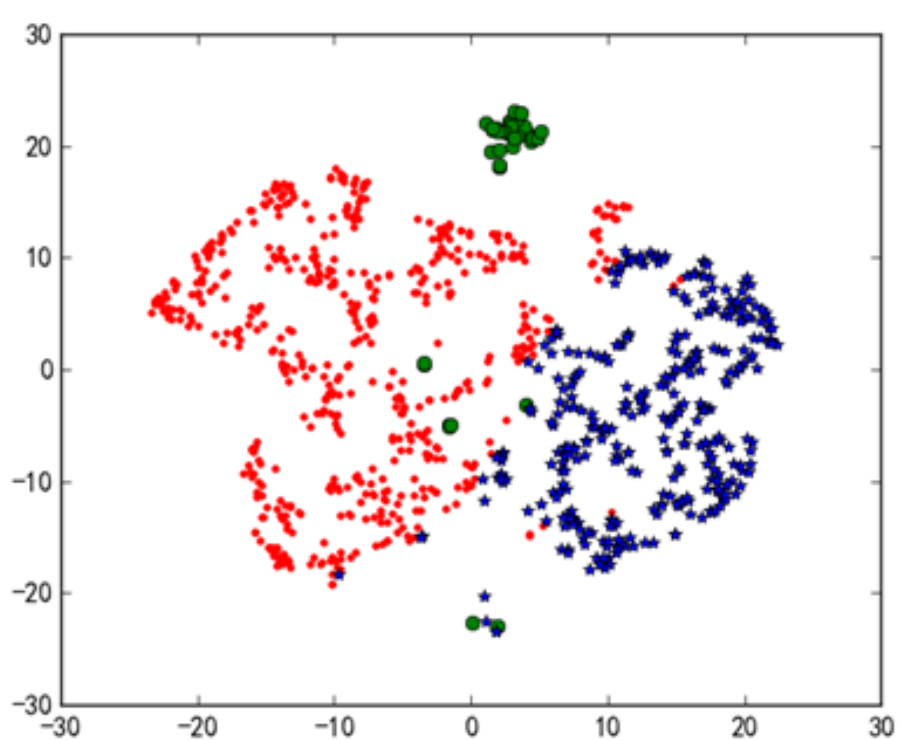
- 1、**降维**过程： $X^{n \times d} \rightarrow W^{n \times n} \rightarrow Y^{n \times k}$ ，再给其他聚类方法进行最终的聚类。如果 $d \gg k$ ，可以视作对原始数据进行降维。
- 2、**表达学习**过程： $W^{n \times n} \rightarrow Y^{n \times k}$ ，再给其他聚类方法进行最终的聚类。可以将Y视作从W中学到节点的表达。

Python聚类方法



聚类分析——聚类结果可视化的工具

- TSNE是Laurens van der Maaten和Geoffrey Hinton在2008年提出的，它的定位是高维数据的可视化。我们总喜欢能够**直观**地展示研究结果，聚类也不例外。然而通常来说输入的特征数是高维的（大于3维），一般难以直接以原特征对聚类结果进行展示。而TSNE提供了一种有效的**数据降维**方式，让我们可以在2维或者3维的空间中展示聚类结果。



聚类分析——Python for data science cheat sheet

Python For Data Science Cheat Sheet

Scikit-Learn

Learn Python for data science interactively at [www.DataCamp.com](https://www.datacamp.com)



Scikit-learn

Scikit-learn is an open source Python library that implements a range of machine learning, preprocessing, cross-validation and visualization algorithms using a unified interface.



A Basic Example

```
>>> from sklearn import neighbors, datasets, preprocessing
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.metrics import accuracy_score
>>> iris = datasets.load_iris()
>>> X, y = iris.data[:, :2], iris.target
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=33)
>>> scaler = preprocessing.StandardScaler().fit(X_train)
>>> X_train = scaler.transform(X_train)
>>> X_test = scaler.transform(X_test)
>>> knn = neighbors.KNeighborsClassifier(n_neighbors=5)
>>> knn.fit(X_train, y_train)
>>> y_pred = knn.predict(X_test)
>>> accuracy_score(y_test, y_pred)
```

Loading The Data

Also see NumPy & Pandas

Your data needs to be numeric and stored as NumPy arrays or SciPy sparse matrices. Other types that are convertible to numeric arrays, such as Pandas DataFrame, are also acceptable.

```
>>> import numpy as np
>>> X = np.random.random((10,5))
>>> y = np.array(['M','M','F','F','M','F','M','M','F','F'])
>>> X[X < 0.7] = 0
```

Training And Test Data

```
>>> from sklearn.model_selection import train_test_split
>>> X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    random_state=0)
```

Preprocessing The Data

Standardization

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler().fit(X_train)
>>> standardized_X = scaler.transform(X_train)
>>> standardized_X_test = scaler.transform(X_test)
```

Normalization

```
>>> from sklearn.preprocessing import Normalizer
>>> scaler = Normalizer().fit(X_train)
>>> normalized_X = scaler.transform(X_train)
>>> normalized_X_test = scaler.transform(X_test)
```

Binarization

```
>>> from sklearn.preprocessing import Binarizer
>>> binarizer = Binarizer(threshold=0.0).fit(X)
>>> binary_X = binarizer.transform(X)
```

Create Your Model

Supervised Learning Estimators

Linear Regression

```
>>> from sklearn.linear_model import LinearRegression
>>> lr = LinearRegression(normalize=True)
```

Support Vector Machines (SVM)

```
>>> from sklearn.svm import SVC
>>> svc = SVC(kernel='linear')
```

Naive Bayes

```
>>> from sklearn.naive_bayes import GaussianNB
>>> gnb = GaussianNB()
```

KNN

```
>>> from sklearn import neighbors
>>> knn = neighbors.KNeighborsClassifier(n_neighbors=5)
```

Unsupervised Learning Estimators

Principal Component Analysis (PCA)

```
>>> from sklearn.decomposition import PCA
>>> pca = PCA(n_components=0.95)
```

K Means

```
>>> from sklearn.cluster import KMeans
>>> k_means = KMeans(n_clusters=3, random_state=0)
```

Model Fitting

Supervised learning

```
>>> lr.fit(X, y)
>>> knn.fit(X_train, y_train)
>>> svc.fit(X_train, y_train)
```

Fit the model to the data

Unsupervised Learning

```
>>> k_means.fit(X_train)
>>> pca_model = pca.fit_transform(X_train)
```

Fit the model to the data
Fit to data, then transform it

Prediction

Supervised Estimators

```
>>> y_pred = svc.predict(np.random.random((2,5)))
>>> y_pred = lr.predict(X_test)
>>> y_pred = knn.predict_proba(X_test)
```

Predict labels
Predict labels
Estimate probability of a label

Unsupervised Estimators

```
>>> y_pred = k_means.predict(X_test)
```

Predict labels in clustering algos

Evaluate Your Model's Performance

Classification Metrics

Accuracy Score

```
>>> knn.score(X_test, y_test)
>>> from sklearn.metrics import accuracy_score
>>> accuracy_score(y_test, y_pred)
```

Estimator score method
Metrics scoring functions

Classification Report

```
>>> from sklearn.metrics import classification_report
>>> print(classification_report(y_test, y_pred))
```

Precision, recall, f1-score and support

Confusion Matrix

```
>>> from sklearn.metrics import confusion_matrix
>>> print(confusion_matrix(y_test, y_pred))
```

Regression Metrics

Mean Absolute Error

```
>>> from sklearn.metrics import mean_absolute_error
>>> y_true = [3, -0.5, 2]
>>> mean_absolute_error(y_true, y_pred)
```

Mean Squared Error

```
>>> from sklearn.metrics import mean_squared_error
>>> mean_squared_error(y_test, y_pred)
```

R² Score

```
>>> from sklearn.metrics import r2_score
>>> r2_score(y_true, y_pred)
```

Clustering Metrics

Adjusted Rand Index

```
>>> from sklearn.metrics import adjusted_rand_score
>>> adjusted_rand_score(y_true, y_pred)
```

Homogeneity

```
>>> from sklearn.metrics import homogeneity_score
>>> homogeneity_score(y_true, y_pred)
```

V-measure

```
>>> from sklearn.metrics import v_measure_score
>>> metrics.v_measure_score(y_true, y_pred)
```

Cross-Validation

```
>>> from sklearn.cross_validation import cross_val_score
>>> print(cross_val_score(knn, X_train, y_train, cv=4))
>>> print(cross_val_score(lr, X, y, cv=2))
```

Tune Your Model

Grid Search

```
>>> from sklearn.grid_search import GridSearchCV
>>> params = {'n_neighbors': np.arange(1,3),
>>>           "metric": ["euclidean", "cityblock"]}
>>> grid = GridSearchCV(estimator=knn,
>>>                     param_grid=params)
>>> grid.fit(X_train, y_train)
>>> print(grid.best_score_)
>>> print(grid.best_estimator_.n_neighbors)
```

Randomized Parameter Optimization

```
>>> from sklearn.grid_search import RandomizedSearchCV
>>> params = {'n_neighbors': range(1,5),
>>>           "weights": ["uniform", "distance"]}
>>> rsearch = RandomizedSearchCV(estimator=knn,
>>>                               param_distributions=params,
>>>                               cv=4,
>>>                               n_iter=8,
>>>                               random_state=5)
>>> rsearch.fit(X_train, y_train)
>>> print(rsearch.best_score_)
```

DataCamp

Learn Python for Data Science interactively



聚类分析——Python for data science cheat sheet

PYTHON FOR DATA SCIENCE CHEAT SHEET

Python Scikit-Learn

Introduction

Scikit-learn: "sklearn" is a machine learning library for the Python programming language. Simple and efficient tool for data mining, Data analysis and Machine Learning.

Importing Convention - import sklearn

Preprocessing

Data Loading

- **Using NumPy:**
>>> import numpy as np
>>> a = np.array([(1,2,3,4),(7,8,9,10)], dtype=int)
>>> data = np.loadtxt('file_name.csv', delimiter=',')
- **Using Pandas:**
>>> import pandas as pd
>>> df = pd.read_csv('file_name.csv', header=0)

Train-Test Data

```
>>> from sklearn.model_selection import train_test_split  
  
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

Data Preparation

- **Standardization**
>>> from sklearn.preprocessing import StandardScaler
>>> get_names = df.columns
>>> scaler = preprocessing.StandardScaler()
>>> scaled_df = scaler.fit_transform(df)
>>> scaled_df = pd.DataFrame(scaled_df, columns=get_names)
- **Normalization**
>>> from sklearn.preprocessing import Normalizer
>>> pd.read_csv('File_name.csv')
>>> x_array = np.array(df['Column1'])
Normalize Column1
>>> preprocessed_normalized_X = preprocessing.normalize([x_array])

Working On Model

Model Choosing

Supervised Learning Estimator:

- **Linear Regression:**
>>> from sklearn.linear_model import LinearRegression
>>> new_lr = LinearRegression(normalize=True)
- **Support Vector Machine:**
>>> from sklearn.svm import SVC
>>> new_svc = SVC(kernel='linear')

Naive Bayes:

```
>>> from sklearn.naive_bayes import GaussianNB  
>>> new_gnb = GaussianNB()  
  
• KNN:  
>>> from sklearn import neighbors  
>>> knn = neighbors.KNeighborsClassifier(n_neighbors=1)
```

Unsupervised Learning Estimator:

- **Principal Component Analysis (PCA):**
>>> from sklearn.decomposition import PCA
>>> new_pca = PCA(n_components=0.95)
- **K Means:**
>>> from sklearn.cluster import KMeans
>>> k_means = KMeans(n_clusters=5, random_state=0)

Train-Test Data

Supervised:

```
>>> new_lr.fit(X, y)  
>>> knn.fit(X_train, y_train)  
>>> new_svc.fit(X_train, y_train)
```

Unsupervised:

```
>>> k_means.fit(X_train)  
>>> pca_model_fit = new_pca.fit_transform(X_train)
```

Post-Processing

Prediction

Supervised:

```
>>> y_predict = new_svc.predict(np.random.random((3,5)))  
>>> y_predict = new_lr.predict(X_test)  
>>> y_predict = knn.predict_proba(X_test)
```

Unsupervised:

```
>>> y_pred = k_means.predict(X_test)
```

Model Tuning

Grid Search:

```
>>> from sklearn.grid_search import GridSearchCV  
>>> params = {"n_neighbors": np.arange(1,3), "metric": ["euclidean", "cityblock"]}  
>>> grid = GridSearchCV(estimator=knn, param_grid=params)  
>>> grid.fit(X_train, y_train)  
>>> print(grid.best_score_)  
>>> print(grid.best_estimator_.n_neighbors)
```

Randomized Parameter Optimization:

```
>>> from sklearn.grid_search import RandomizedSearchCV  
>>> params = {"n_neighbors": range(1,5), "weights": ["uniform", "distance"]}  
>>> rsearch = RandomizedSearchCV(estimator=knn, param_distributions=params, cv=4, n_iter=8, random_state=5)  
>>> rsearch.fit(X_train, y_train)  
>>> print(rsearch.best_score_)
```

Evaluate Performance

Classification:

1. Confusion Matrix:

```
>>> from sklearn.metrics import confusion_matrix  
>>> print(confusion_matrix(y_test, y_pred))
```

2. Accuracy Score:

```
>>> knn.score(X_test, y_test)  
>>> from sklearn.metrics import accuracy_score  
>>> accuracy_score(y_test, y_pred)
```

Regression:

1. Mean Absolute Error:

```
>>> from sklearn.metrics import mean_absolute_error
```

```
>>> y_true = [3, -0.5, 2]  
>>> mean_absolute_error(y_true, y_predict)
```

2. Mean Squared Error:

```
>>> from sklearn.metrics import mean_squared_error  
>>> mean_squared_error(y_test, y_predict)
```

3. R² Score:

```
>>> from sklearn.metrics import r2_score  
>>> r2_score(y_true, y_predict)
```

Clustering:

1. Homogeneity:

```
>>> from sklearn.metrics import homogeneity_score  
>>> homogeneity_score(y_true, y_predict)
```

2. V-measure:

```
>>> from sklearn.metrics import v_measure_score  
>>> metrics.v_measure_score(y_true, y_predict)
```

Cross-validation:

```
>>> from sklearn.cross_validation import cross_val_score  
>>> print(cross_val_score(knn, X_train, y_train, cv=4))  
>>> print(cross_val_score(new_lr, X, y, cv=2))
```

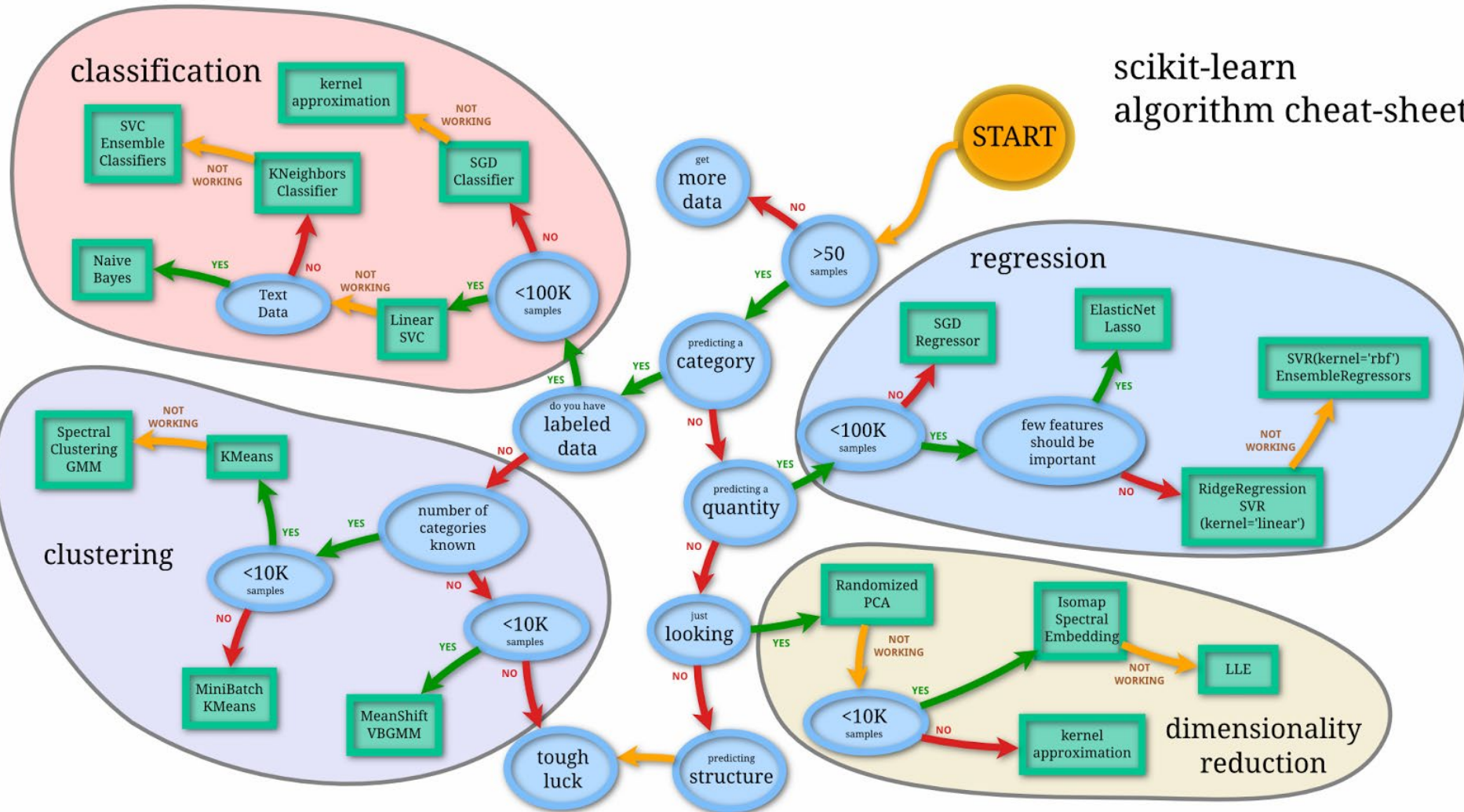
FURTHERMORE:

Python for Data Science Certification Training Course



聚类分析——Scikit-learn algorithm cheat-sheet

scikit-learn
algorithm cheat-sheet



Thank You!